Name:

GCSE COMPUTER SCIENCE

COMP2 Revision Booklet

Your Essential Exam Revision Guide

Welcome to your revision guide. It will help guide you through the key ideas you *really* need to know. You'll find clear explanations, helpful reminders, and practice questions to train your brain step by step.

Use this booklet little and often to organise your thinking, grow your confidence, and walk calmly into your exams knowing you're ready.

Paper 2 Exam Date:
Summer 2027 - check your exam timetable

Contents

2.1 Algorithms....2
2.1.1 Computational Thinking....3
2.1.1 Exam questions on Computational Thinking....4
2.1.2 Exam questions on Designing, Creating and Refining Algorithms....10
2.1.3 Searching and Sorting Algorithms....12
2.1.3 Exam Questions on Searching and Sorting Algorithms....15
2.2 Programming Fundamentals....16
2.2.1 Programming Fundamentals....19
2.2.1 Exam Questions on Programming Fundamentals....22
2.2.2 Data Types....23
2.2.2 Exam Questions on Data Types....26
2.2.3 Additional Programming Techniques....28
2.2.3 String Handling....28
2.2.3 One Dimensional Arrays....29
2.2.3 Two Dimensional Arrays....30
2.2.3 File Handling....32
2.2.3 SQL....33
2.2.3 2D arrays to emulate SQL....34
2.2.3 Subprograms....35
2.2.3 Random numbers....36
2.2.3 Exam Questions on Additional Programming Techniques....37
2.3 Producing Robust Programs....40
2.3.1 Defensive Design....41
2.3.1 Exam Questions on Defensive Design....44
2.3.2 Testing....46
2.3.2 Exam Questions on Testing....49
2.4 Boolean Logic....51
2.4.1 Boolean Logic....52
2.4.1 Exam Questions on Boolean Logic....53
2.5.1 Languages....57
2.5.1 Exam Questions on Languages....60
2.5.2 The Integrated Development Environment (IDE)....61
2.5.2 Questions on the Integrated Development Environment (IDE)....63
Resources....64
Common Exam Questions – worked through....64
AI PREDICTED Exam Paper 2 2026....69
Answers and Common misconceptions....78
AI-PREDICTED Exam Paper 2 – 2026 (Version B)....79
Answers and Common misconceptions....87
Useful QR Codes....89

2.1 Algorithms

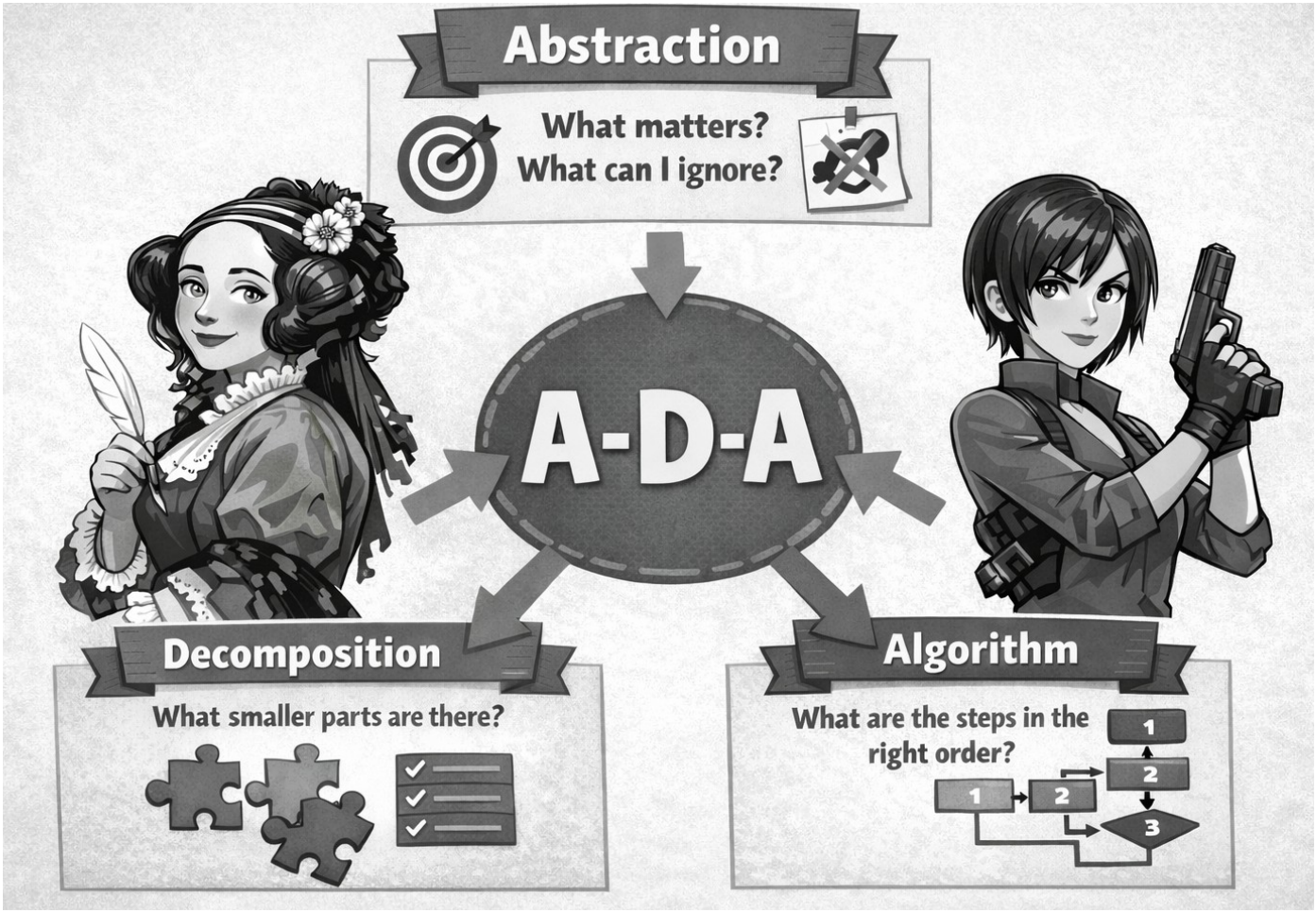
Keyword	Definition
Computational Thinking	The process of thinking about a problem using computational means in order to create a solution which a computer can implement.
Abstraction	The process of removing irrelevant or unnecessary information from the problem in order to better understand the basic parts of it.
Decomposition	The process of breaking a problem down into smaller parts to make it easier to solve.
Algorithmic Thinking	The process of breaking a problem down into smaller parts to make it easier to solve.
Algorithms	A sequence of logical instructions to carry out a particular task.
Structure Diagram	A diagram which shows in graphical format the steps needed to solve a problem. There are several different types of structure diagram, and a flow chart is one example.
Pseudocode	A way of writing out algorithms or other computer programs in a way which sits somewhere between full sentences and computer code.
Flowchart	A way of using pictures and shapes to represent the steps needed to solve a problem or that a program will take.
Programming Language	A language which can be understood by computers to produce computer software.
Trace Table	A table which records the values stored in each of a program's variables at each stage through the program. It is a useful tool when trying to find errors in the code.
Search	Finding a value or piece of information within a set of data.
Binary Search	A way of searching an ordered list by looking at the middle item in the list, then looking at either the items above or below depending on if the required value is higher or lower. This process repeats until the value is found.
Linear Search	A way of searching through a list items, checking them one at a time until the required value is found.
Sort	A way of placing information into a certain order, such as alphabetically or by number.
Bubble Sort	A way or sorting a list by comparing the items in a list, two at a time, and changing the order if needed. The process repeats until the list is in the correct order.
Merge Sort	A way of sorting a list by dividing it in two until each item is separated individually. Pairs of items are compared and placed in order until the list as a whole is ordered.
Insertion Sort	A way of sorting a list by working through the items one at a time and placing them in the correct position in the list.

2.1.1 Computational Thinking

2.1.1 Computational thinking	
☐ Principles of computational thinking: ○ Abstraction ○ Decomposition ○ Algorithmic thinking	**Required** ✓ Understanding of these principles and how they are used to define and refine problems

Computational thinking is a way of solving problems by breaking them into smaller parts, ignoring unimportant details, and writing clear step-by-step instructions, for example, planning a GCSE revision timetable: list subjects → split each into topics → pick the most important areas → plan short revision sessions → test yourself and adjust each week.

Principle	What it means (simple)	Why it helps	Quick example
Abstraction	Keep the **important** details, ignore what you **don't need**	Makes the problem simpler and clearer	Bus app: keep stop names/times, ignore seat colour
Decomposition	Break a big problem into **smaller tasks**	Easier to solve, build, and test	Game: movement, scoring, collisions, menu
Algorithmic thinking	Plan **step-by-step instructions** in the right order	A computer can follow the steps exactly	Login: input user/pass → check → allow/deny



Quick exam sentence starters

- **Abstraction:** "I will ignore unnecessary details like… and keep…"
- **Decomposition:** "I will split the problem into…"
- **Algorithmic thinking:** "I will create steps in order: first…, then…, next…

2.1.1 Exam questions on Computational Thinking

Question	Tip to answer	Common misconception
1. Which option is the best example of **decomposition**? A) Ignoring the colour of a phone case B) Splitting a game into levels, scoring and controls C) Writing code in Python D)Using Wi-Fi to connect devices **(1)** B	**Tip:** Decomposition = split it up.	✘ Picking abstraction (ignoring detail).
2. Which statement describes **abstraction**? A) Breaking a problem into smaller parts B) Testing a program C) Keeping important details and ignoring the rest D) Writing steps in order **(1)** C	**Tip:** Abstraction = ignore the fluff.	✘ Mixing it up with decomposition.
3. What is meant by **computational thinking**? **(1)** Computational thinking is a way to solve problems by thinking in clear, logical steps.	**Tip:** Use "solve problems" + "logical steps".	✘ Talking only about coding.
4. What is an **algorithm**? **(1)** An algorithm is a set of step-by-step instructions to solve a problem.	**Tip:** Say step-by-step instructions.	✘ Calling it "code".
5. State **two** computational thinking techniques. **(2)** Abstraction.$_1$ Decomposition.$_2$	**Tip:** Two techniques = two marks.	✘ Giving examples instead of naming them.
6. State **two** features of a problem that make it **computable**. **(2)** You can write clear step-by-step instructions (an algorithm).$_1$ The steps are precise and repeatable.$_2$	**Tip:** Think: can I write exact steps?	✘ Saying "needs a computer".
7. A school is making a **lunch pre-order app**. Explain how **abstraction** helps when planning the app. **(3)**	**Tip:** Say what you keep + what you ignore.	✘ Listing features with no explanation.

Abstraction keeps only important information.[1] Keep meal choice/allergies/payment.[2] Ignore details that don't affect ordering.[3]

---

8. A student is designing a **multiplayer racing game**. Explain how **decomposition**helps. **(3)**

Split into parts like controls, lap counting and leaderboard.[1] Build each part separately.[2] Then combine into one full game.[3]

Tip: Name 3 mini-tasks.

✘ Only saying "it makes it easier".

---

9. Explain why **not all problems** can be solved by a computer. **(3)**

A computer needs an algorithm (clear steps).[1] Some problems can't be written as exact steps.[2] So a computer can't reliably solve them.[3]

Tip: Use the word algorithm.

✘ "Computers aren't smart" (too vague).

---

10. Match each technique to the best example. Techniques: Abstraction / Decomposition / Algorithmic thinking.

Examples:
A) Plan the exact order of steps for a password reset
B) Split a social media app into feed, messages and settings
C) Ignore profile picture style and keep username + age only

(3)

Algorithmic thinking → A.[1] Decomposition → B.[2] Abstraction → C.[3]

Tip: One technique per example.

✘ Linking more than once.

---

11. Compare **abstraction** and **decomposition**. **(4)**

Both make problems easier.[1] Abstraction removes unneeded detail.[2] Decomposition splits into smaller tasks.[3] Example given in context.[4]

Tip: Same + different + example.

✘ Only describing one technique.

---

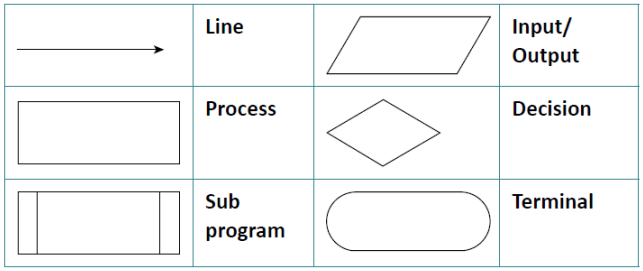
12. A college wants a system that tracks **attendance** and messages home if it drops. Explain how computational thinking techniques help you **define and refine** the problem. **(6)**

Decompose into register/store/calculate/message.[1] Abstract to keep only needed data.[2] Set clear rules (e.g. threshold).[3] Write ordered steps for the process.[4] Makes testing easier.[5] Final solution is clearer and more reliable.[6]

Tip: 3 short chunks: Decomposition, Abstraction, Algorithmic thinking.

✘ Forgetting to refine rules/inputs/outputs.

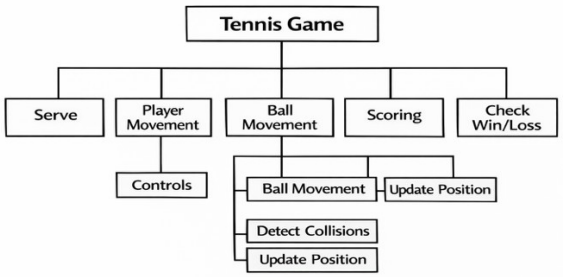
2.1.2 Designing, Creating and Refining Algorithms

2.1.2 Designing, creating and refining algorithms	
☐ Identify the inputs, processes, and outputs for a problem ☐ Structure diagrams ☐ Create, interpret, correct, complete, and refine algorithms using: ○ Pseudocode ○ Flowcharts ○ Reference language/high-level programming language ☐ Identify common errors ☐ Trace tables	**Required** ✓ Produce simple diagrams to show: ▪ The structure of a problem ▪ Subsections and their links to other subsections ✓ Complete, write or refine an algorithm using the techniques listed ✓ Identify syntax/logic errors in code and suggest fixes ✓ Create and use trace tables to follow an algorithm ✓ Use of nesting for selection and iteration **Flowchart symbols** 

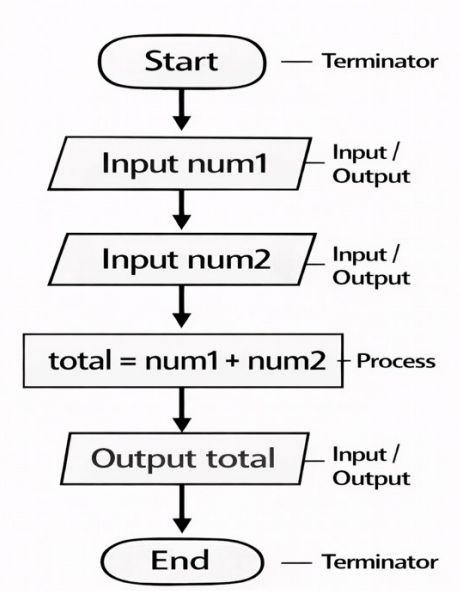
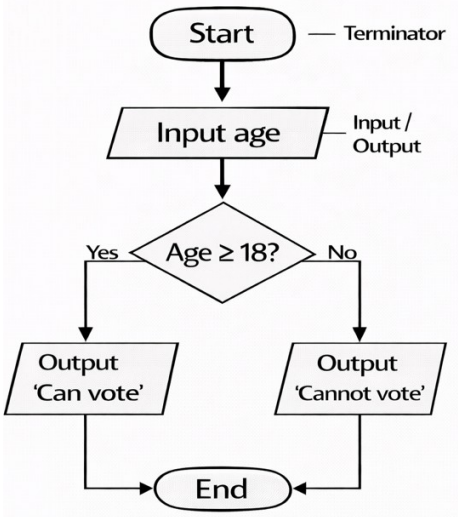
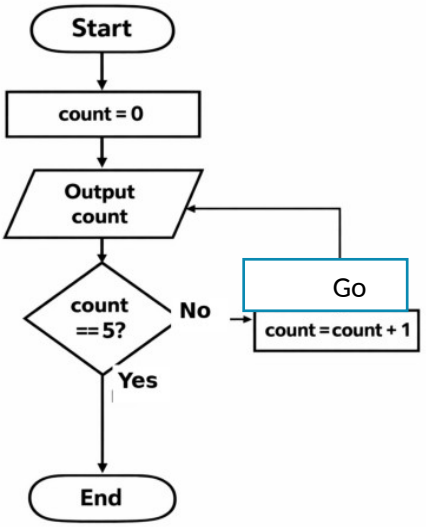
An algorithm is set of instructions used to solve a problem. It is **designed** by identifying the inputs, the processes needed to manipulate the data, and the outputs produced by the program.

Stage	What it means (simple)	Common examples	Exam tip
Input	Data that goes **into** the program	User enters a number Username / password / Data read from a file	Look for `input()` or file reading
Process	What the program **does** with the data	Calculations IF statements Loops (FOR / WHILE) Searching or sorting	Use selection and iteration
Output	The result the program **produces**	Message on screen Total or result Data written to a file	Usually uses `print()`

Structure Diagram

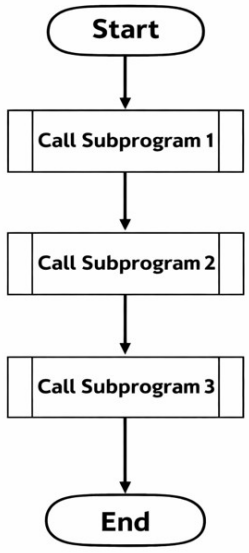


Reason	Why it matters
Planning	Makes complex problems easier
Decomposition	Breaks big problems into parts
Coding	Each section can become a procedure or function
Debugging	Errors are easier to find
Exams!	Often linked to 3–6 mark questions

Scenario	Flowchart	Pseudocode
Sequence: ***Add two numbers and output the total***		num1 = input("Enter first number") num2 = input("Enter second number") num3 = input("Enter third number") total = num1 + num2 + num3 print(total) (*Exam note - no datatypes in Pseudocode)*
Selection ***Check if someone is old enough to vote (18+)*** Decision (Yes or No)		age = input("Enter age") **if** age >= 18 **then** print("Can vote") **else** print("Too young") endif *(Exam note - no datatypes in Pseudocode)*
Iteration ***Count from 0 to 5 and output each number*** Go Back - That's iteration !		for count =0 to 5 print(count) next count

Subprograms

Subprograms are named blocks of code that do one specific job, and only run when they are **called**.



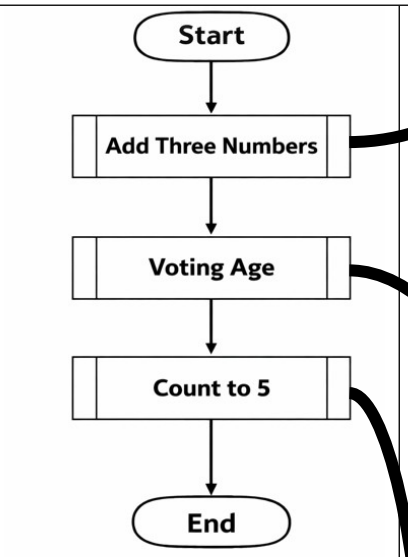
Why use subprograms?
Make programs **shorter and easier to read**
Avoid **repeating** the same code
Make testing and fixing bugs **easier** (you can test one part at a time)

Two types you need to know

Procedure
Runs code but **does not return** a value
Used for tasks like displaying a menu, printing a message, saving to a file

Function
Runs code and **returns** a value back to the main program
Used for calculations like total price, average score, converting units.

More of this in Section 2.2!

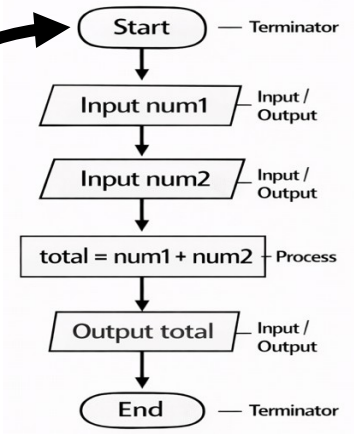


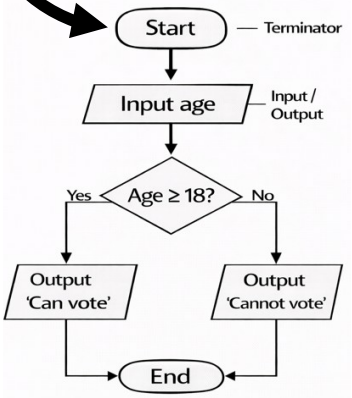

```
Procedure
AddThreeNumbers()
    num1 = input("Enter first
        number")
    num2 = input("Enter second
        number")
    num3 = input("Enter third
        number")
    total = num1 +  num2 +
        num3
    print(total)
Endprocedure

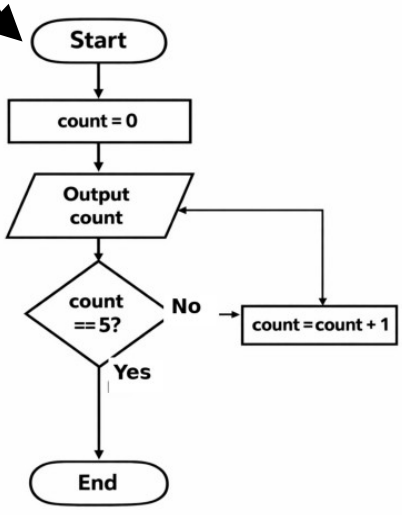
Procedure VotingAge()
    age = input("Enter age")
    if age >= 18 then
        print("Can vote")
    else
        print("Too young")
    endif
Endprocedure

Procedure CountTo5()
    for count =0 to 5
        print(count)
    next count
Endprocedure

//Main
    AddThreeNumbers()
    VotingAge()
    CountTo5()
```

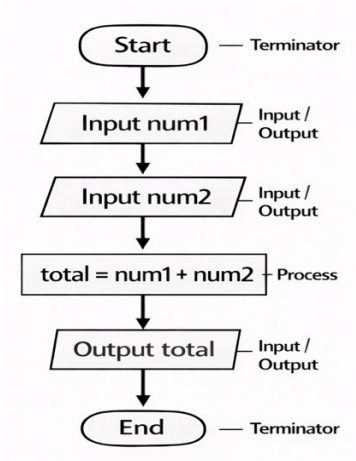






A **trace table** is a simple table you fill in to **track what happens to variables step by step** as an algorithm runs.Used to **check your logic works** (spot mistakes early)

Flowchart



Pseudocode

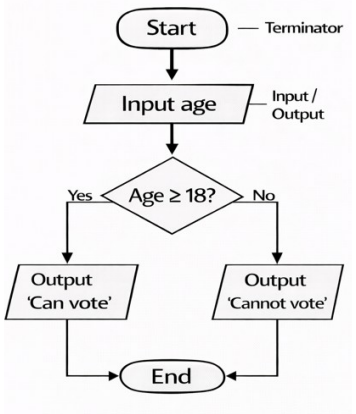
Line	Code
1	num1 = input("Enter first number")
2	input("Enter second number")
3	total = num1 + num2
4	print(total)

Trace Table

User enters 6 then 7

Line	num1	num2	total	Output
1	6			
2	6	7		
3	6	7	13	
4	6	7	13	13

Flowchart



Pseudocode

Line	Code
1	age = input("Enter age")
2	If age>=18 then
3	print("Can vote")
4	**else**
5	print("Too young")
6	endif

Trace Table

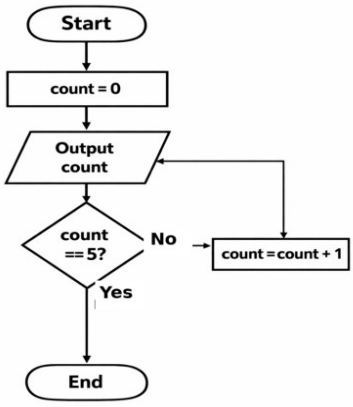
User enters 3

Line	age	Output
1	3	
2		
5		"Too young")

User enters 18

Line	age	Output
1	18	
2		
3		"Can Vote")

Flowchart



Pseudocode

Line	Code
1	for count =0 to 5
2	print(count)
3	next count

Trace Table

Line	count	Output
1	1	
2		1
3	2	
2		2
3	3	
2		3
3	4	
2		4
3	5	
2		5
3	6	

Trace an iteration – worked method: For x = 8, the statements print("GO"); x = x - 3 repeat while x > 0. Track the value before each output: 8, 5, 2; the message appears three times. Always check whether the condition is tested before or after the loop body.

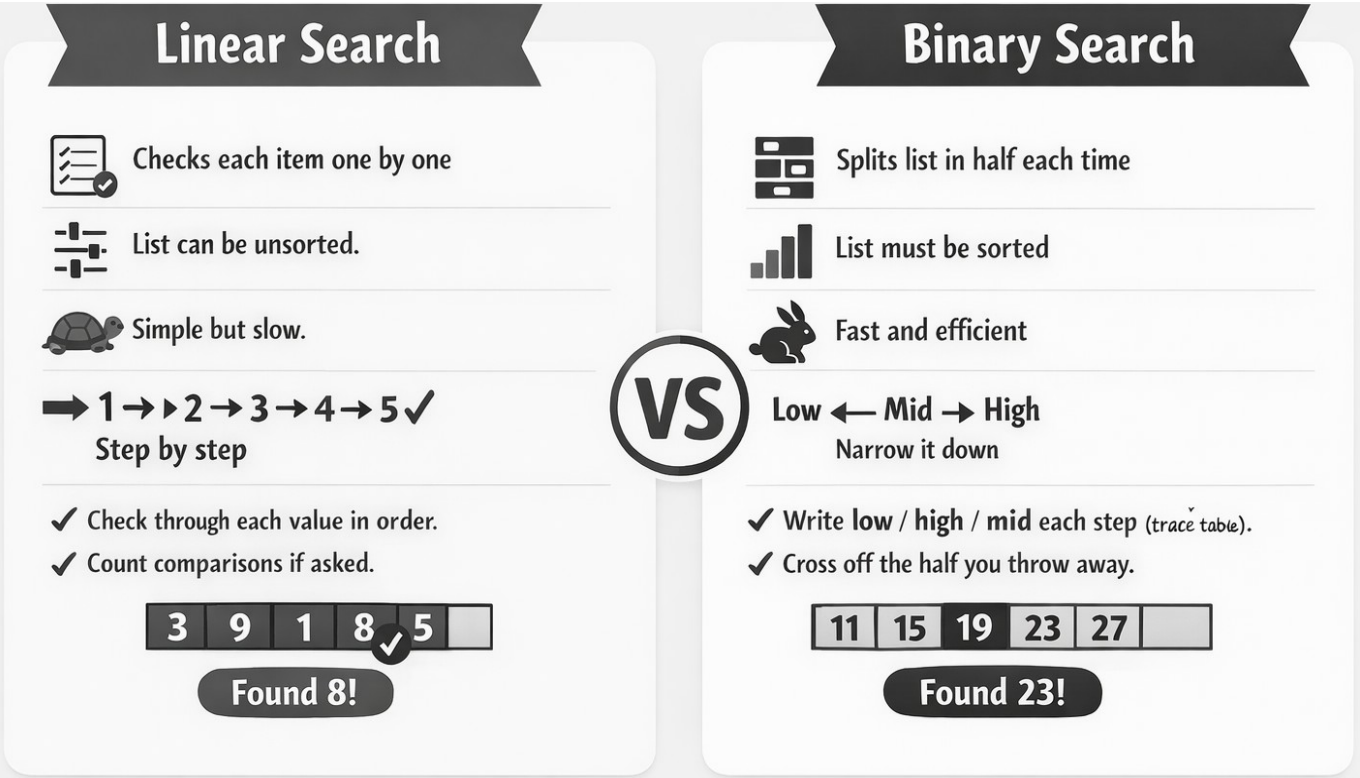
2.1.2 Exam questions on Designing, Creating and Refining Algorithms

Question	Tip to answer (student)
1. Which method is BEST for writing an algorithm that is easy for any programmer to understand? (Tick one) (1) ☐ Everyday English ☐ Pseudocode ☐ A specific programming language ☐ Machine code Pseudocode (1)	Think: program language independent.
2. A programmer writes an algorithm in everyday English. Explain one problem with this. (2) Different people may understand the steps differently.[1] This can lead to different outcomes or mistakes.[2]	"Open to interpretation" = key phrase.
3. State **two** reasons why writing an algorithm in a specific programming language can be a bad idea at the design stage. (2) It is tied to one language (not portable).[1] It can be slower to write than pseudocode/flowcharts.[2]	Two separate reasons.
4. Name **two** ways algorithms are commonly represented (2) Flowcharts[1] and pseudocode.[2]	Just name both.
5. Complete the missing word: "Pseudocode is _____ dependent and uses no fixed syntax." (1) language [1]	OCR phrase.
6. Identify the correct flowchart symbol for a decision. (Tick one) ☐ Rectangle ☐ Parallelogram ☐ Diamond ☐ Oval (1) Diamond [1]	Decision = question/IF.
7. A flowchart contains an input box and an output box. Describe what each one does. (4) Input box: gets data from the user/sensor/file.[1] It stores the value in a variable for the program to use.[2] Output box: shows results to the user (screen/sound/print).[3] It displays the value/message produced by the algorithm.[4]	2 marks each idea: input vs output.
8. A student is asked to "correct an algorithm". What does this usually mean? (2) Find an error in the steps/logic.[1] Fix it so the algorithm works as intended.	Mention logic / missing steps.

Question	Tip to answer (student)
9. A program should only accept a username that is **6 to 12 characters** long. Complete the condition: IF length(username) ____ 6 AND length(username) ____ 12 THEN (2) >= 6[1] <= 12[2]	Use >= and <= (inclusive).
10. A programmer makes this mistake: IF score > 10 AND < 20 THEN Explain why this is incorrect and give the correct version. (3) The second part is not a full comparison (it does not include the variable).[1] You must compare score in both conditions.[2] Correct: IF score > 10 AND score < 20 THEN.[3]	Must compare the variable twice.
11. Describe how a **trace table** helps when refining an algorithm. (3) You follow the algorithm step by step.[1] You record how variables change after each step.[2] This helps spot where the logic goes wrong so it can be fixed.[3]	Mention values changing + finding bugs.
12. A dry run is carried out on an algorithm. What is a dry run? (2) Testing the algorithm without coding it.[1] You step through the logic using sample inputs to check outputs.[2]	"Run it in your head".
13. A flowchart is incomplete. State **two** things you should check before adding the missing symbols. (2) Follow the arrows to understand the current logic.[1] Check the meaning of symbols so you add the correct type (process/decision/I/O).[2]	Two checks.
14. A programmer is designing an algorithm for a school canteen system. They start coding immediately and do not plan the algorithm first. Explain **two** problems this could cause. (4) They may misunderstand the problem and miss key requirements.[1] This leads to a solution that does not match what is needed.[2] They are more likely to make logic errors because steps are not planned.[3] Fixing mistakes later can take longer than planning first.[4]	Think: bugs + missed requirements + messy code.

2.1.3 Searching and Sorting Algorithms

2.1.3 Searching and sorting algorithms	
☐ Standard searching algorithms: ○ Binary search ○ Linear search ☐ Standard sorting algorithms: ○ Bubble sort ○ Merge sort ○ Insertion sort	**Required** ✓ Understand the main steps of the algorithm and the segments of code in it ✓ Understand any pre-requisites of an algorithm ✓ Apply the algorithm to a data set ✓ Identify an algorithm if given the code, pseudocode or Exam Reference Language for it **Not required** × To remember the code, pseudocode or Exam Reference Language for these algorithms



	Linear search (simple scan)	Binary search (split in half)
What it does	Checks items **one by one** until it finds the target (or reaches the end).	Repeatedly checks the **middle** and halves the search area.
Pre-requisite	**None** (list can be unsorted).	List must be **sorted** (usually ascending).
Main steps (in words)	1) Start at the first item. 2) Compare to target. 3) If match → **found**. 4) If not, move to next. 5) End reached → **not found**.	1) Set **low** and **high**. 2) Find **mid**. 3) Compare `list[mid]` to target: equal → found; bigger → right half (`low = mid + 1`); smaller → left half (`high = mid - 1`). 4) Repeat until found or `low > high`.
How to apply to a data set	Tick through each value in order. Count comparisons if asked.	Record **low / high / mid** each step (trace table). Cross off the half you throw away.
How to spot it in code / pseudocode	A loop that goes through the list from start to end (`FOR each item` / index +1). Often stops early if found.	Uses **low, high, mid**. Often `WHILE low <= high`. Updates `low = mid + 1` or `high = mid - 1`.
Mini example	Find 8 in `[3, 9, 1, 8, 5]` → 3 ❌, 9 ❌, 1 ❌, 8 ✅ found.	Find 23 in `[11, 15, 19, 23, 27]` → mid=19 → go right `[23, 27]` → mid=23 ✅ found.

BMI – Bubble Merge Insertion

Bubble Sort

Sort [5, 1, 4, 2] and show each pass.

Pass	Start of pass	Compare & swap steps (left → right)	End of pass
1	5 1 4 2	Compare 5 & 1 → swap Compare 5 & 4 → swap Compare 5 & 2 → swap	1 4 2 5
2	1 4 2 5	Compare 1 & 4 → no swap Compare 4 & 2 → swap Compare 4 & 5 → no swap	1 4 2 5 1 2 4 5
3	1 4 2 5	Compare 1 & 2 → no swap Compare 4 & 4 → no swap Compare 4 & 5 → no swap	1 2 4 5 1 2 4 5
3	1 2 4 5	Compare 1 & 2 → no swap Compare 2 & 4 → no swap Compare 4 & 5 → no swap	1 2 4 5 1 2 4 5 ✓

```
// Bubble Sort (simple pseudocode)
REPEAT
    swapped ← FALSE

    FOR i ← 0 TO length(list) - 2
        IF list[i] > list[i + 1] THEN
            temp ← list[i]
            list[i] ← list[i + 1]
            list[i + 1] ← temp
            swapped ← TRUE
        ENDIF
    NEXT i

UNTIL swapped = FALSE
```

Merge Sort

Sort [5, 1, 4, 2]

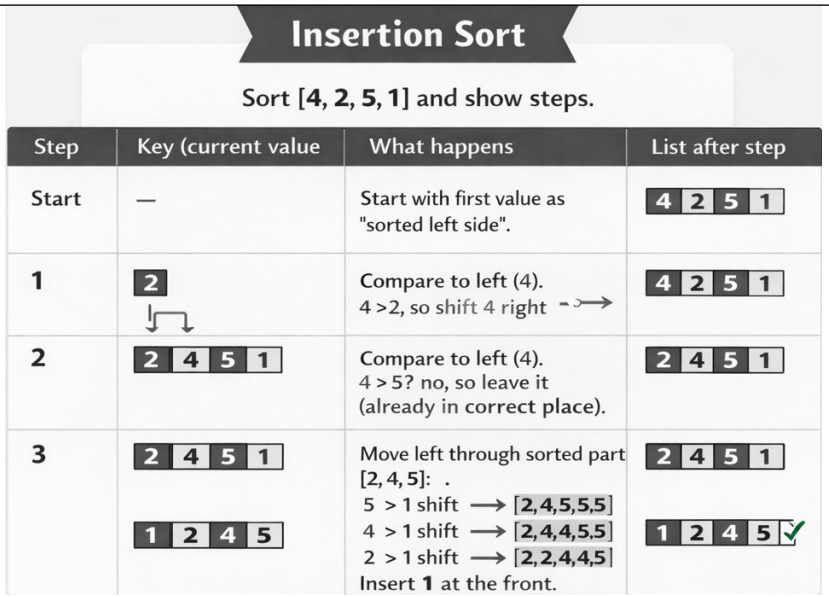
Stage	What you do	Result
Start	Original list	5 1 4 2
Split 1	Split into two halves	5 1 • → 4 2
Split 2	Split each half into single items	5 1 4 2
Merge (left half)	Merge [5] and [1] (take smallest first) ·Compare 5 vs 1 → take 1, then 5	1 5 1, 5
Merge (right half)	Merge [4] and [2] ·Compare 4 vs 2 → take 2, then 4	1 2 4 2, 4
Final merge	Merge [1,5] and [2,4] ·Compare 1 vs 2 → take 1 → [1] ·Compare 5 vs 2 → take 2 → [1, 2] ·Compare 5 vs 4 → take 4 → [1, 2, 4] ·Take remaining 5 →	1 1 2 4 5 1 2, 4 5 1 2, 4 5 5 ✓

```
FUNCTION Merge(left, right)
    result ← empty list

WHILE left not empty AND right
                                not
                                empty
        IF first(left) <= first(right) THEN
            add first(left) to result
            remove first(left) from left
        ELSE
            add first(right) to result
            remove first(right) from
right
        ENDIF
    ENDWHILE

        add remaining items in left to
result
        add remaining items in right to
result

        RETURN result
    ENDFUNCTION
```



Insertion Sort

Sort [4, 2, 5, 1] and show steps.

Step	Key (current value	What happens	List after step
Start	—	Start with first value as "sorted left side".	4 2 5 1
1	2	Compare to left (4). 4 > 2, so shift 4 right →	4 2 5 1
2	2 4 5 1	Compare to left (4). 4 > 5? no, so leave it (already in correct place).	2 4 5 1
3	2 4 5 1 1 2 4 5	Move left through sorted part [2, 4, 5]: . 5 > 1 shift → [2,4,5,5] 4 > 1 shift → [2,4,4,5] 2 > 1 shift → [2,2,4,5] Insert **1** at the front.	2 4 5 1 1 2 4 5 ✓

```
// Insertion Sort pseudocode

FOR i ← 1 TO LENGTH(list) - 1
    key ← list[i]
    j ← i - 1

    WHILE j >= 0 AND list[j] > key
        list[j + 1] ← list[j]     // shift right
        j ← j - 1
    ENDWHILE

    list[j + 1] ← key          // insert key
NEXT i
```

	Bubble sort	Insertion sort	Merge sort
What it does	Compares **neighbours** and **swaps** if wrong order. Biggest values "bubble" to the end.	Builds a sorted section on the **left** by taking a **key** value and inserting it in the right place.	**Splits** the list into halves, then **merges** back in sorted order.
Main steps	Do a pass left→right: compare `i` and `i+1`, swap if needed. Repeat passes until no swaps	Assume first item is sorted. Take next item as **key**. Shift bigger items right. Insert key. Repeat.	Split list in half repeatedly until single items. Merge pairs by repeatedly taking the smallest front item.
Apply to data	Show each **pass** and the swaps. After each pass, the last item is in the correct place.	Show the "sorted left side" growing. Show **shifts** then where the key is placed.	Draw the **splits** (like a tree), then show the **merges** step-by-step.
Spot in code / pseudocode	Compares neighbours (`list[j]` vs `list[j+1]`). Often nested loops or a `swapped` flag.	Uses `key` (or current). Has a loop moving left while `list[j] > key`, shifting (`list[j+1] = list[j]`).	Mentions split into left/right, often recursion, and a clear **merge** step combining two sorted lists.
Mini example idea	`[5,1,4,2]` one pass → `[1,4,2,5]`(5 bubbled to end)	`[4,2,5,1]` → `[2,4]` → `[2,4,5]`→ `[1,2,4,5]`	Merge `[2,6]` and `[1,5]` → `[1,2,5,6]`

Bubble - Go along the list comparing **neighbours.** If two are in the wrong order, **swap** them. Keep doing passes until a full pass makes **no swaps** (then it's sorted).

Merge - Split the list in half again and again until you have single items. Then **merge** the pieces back together by always taking the **smallest front item** first. Merge pairs of **1**, then **2**, then **4**... items until the whole list is sorted.

Insertion - **works by splitting the list into a sorted and an unsorted part.**
One value at a time is taken from the unsorted list and inserted into the correct position in the sorted list by shifting larger values to the right until



a gap is created.

Merge sort – show every stage: [8,3,6,1,7,2,5,4] → split into [8,3,6,1] and [7,2,5,4] → then pairs → single values. Merge sorted pairs [3,8], [1,6], [2,7], [4,5]; merge halves [1,3,6,8], [2,4,5,7]; final [1,2,3,4,5,6,7,8]. Credit comes from showing the process, not just the final answer.

"The more we order the world, the more clearly it speaks to us."

— ***Sister Teresa / Pierre Teilhard de Chardin***

2.1.3 Exam Questions on Searching and Sorting Algorithms

Question	Tip to answer (student)
1. Which search algorithm **requires the data to be in order**? (Tick one) (1) ☐ Linear search ☐ Binary search Binary search [1]	Ordered list = binary.
2. State one condition that will stop a **linear search**. (1) The item is found **or** the end of the list is reached. [1]	Think: found OR end.
3. A student uses **binary search** on an unordered list. Explain why this can fail. (2) Binary search assumes the data is sorted.[1] If it isn't, the algorithm may discard the half containing the value.[2]	Mention "wrong half".
4. A list has 12 items. Explain why binary search is usually faster than linear search on large ordered lists. (3) Binary search checks the middle value first.[1] It throws away half the list each step.[2] So it needs fewer comparisons than checking every item one by one.[3]	Use "halves each time".
5. Complete the statement: "Linear search checks items _____ until the value is found." (1) one-by-one [1]	Simple word.
6. A bubble sort compares pairs of values and swaps them when they are... (Tick one) (1) ☐ in the correct order ☐ in the incorrect order ☐ always swapped in the incorrect order [1]	Bubble = swap wrong pairs.
7. A bubble sort makes a "pass" through a list. Describe what happens in one pass. (2) The algorithm compares each pair of neighbours.[1] It swaps them if they are in the wrong order.[2]	Compare + swap.
8. The data is: **[5, 1, 4, 2]**. Show the list **after one full pass** of bubble sort (ascending). (3) Compare 5 & 1 → swap → [1,5,4,2][1] Compare 5 & 4 → swap → [1,4,5,2][2] Compare 5 & 2 → swap → [1,4,2,5][3]	Swap neighbours left to right.
9. Explain why bubble sort may still do one extra pass even when the list looks sorted. (2) The algorithm can't "know" it is sorted.[1] It must complete a pass where **no swaps happen** to confirm it.[2]	Needs "no swaps" check.
10. An insertion sort works by keeping a _____ part and an _____ part of the list. (2) sorted[1] and unsorted[2]	OCR wording.
11. Describe one way insertion sort is different from bubble sort. (2) Insertion sort takes the next item and inserts it into the correct place	Pick one clear difference.

Question	Tip to answer (student)

in the sorted part.[1] Bubble sort repeatedly swaps neighbouring items during passes.[2]

12. A merge sort uses "divide and conquer".
Describe what this means. (3)

Split then merge in order.

The list is split into smaller lists.[1] These lists are sorted by merging them back together in order.[2] The process continues until one sorted list remains.[3]

13. Show the **main steps** a merge sort would take to sort: **[7, 2, 9, 1]** into ascending order. (4)

Split → pairs → merge.

Split into single items: [7] [2] [9] [1][1] Merge into sorted pairs: [2,7] and [1,9][2] Merge into one sorted list: [1,2,7,9][3] Clear "divide then merge" steps shown.[4]

14. A searching algorithm checks the first item, then the next, and continues until it finds the value. Name the algorithm. (1)

First → next → next.

Linear search [1]

15. A binary search is being done on this ordered list: **[3, 8, 12, 19, 25, 31, 40]**. The value searched for is **25**.
State the middle value checked first and what happens next. (3)

Middle first, then half.

Middle value is 19.[1] 25 is bigger than 19 so search the right half.[2] New range becomes [25,31,40].[3]

16. A programmer says: "Binary search is always better than linear search."
Give one situation where **linear**search may be more suitable. (2)

Unsorted or tiny list.

If the list is not sorted, linear search works without sorting first.[1] For very small lists, linear search can be simpler and fast enough.[2]

17. **Complete the algorithm** for a linear search that outputs "FOUND" if value is in the list, otherwise "NOT FOUND". (6)

Fill the blanks only.

```
value = input()
found = FALSE
for i = 0 to length(list)-1
   if list[i] == _______ then
      found = _______
   endif
next i
if found == TRUE then
   print("_________")
else
   print("_____________")
endif
```

value[1] TRUE[2] FOUND[3] NOT FOUND[4] Correct comparisons and outputs used.[5] Uses found flag to decide final output.[6]

2.2 Programming Fundamentals

Keyword	Definition
Variable	A named value in which data can be stored by a computer program. The data can be changed as the program runs.
Constant	A named value which is accessed by a computer program but is fixed and cannot be changed.
Operator	Operators compare or modify values in computer code. Examples include + < and NOT.
Input	Data or information which is put into the program. This could be entered via a user or taken from a sensor.
Output	Data or information which the program produces. This could be shown on a screen, sent to a printer, or saved to a file.
Assignment	The process of placing a value into a variable.
Sequence	Following through a set of steps in a particular order.
Selection	Taking different paths through the code depending on a condition.
Iteration	Repeating a set of steps several times.
Count Controlled Loop	Using a counter to loop through one or more lines of code until they have been run a certain number of times.
Condition Controlled Loop	Using a loop to repeat one or more lines of code until a certain condition is met.
Arithmetic Operator	An operator which uses maths to modify numbers such as +
Boolean Operator	An operator which uses a logical criteria to compare two values such as < or AND
Integer	A whole number without a decimal point or fraction. E.g. 1, 100 or 88122
Real	A number which may include a decimal point such as 11.2
Boolean	A value which can be only TRUE or FALSE.
Character	A single symbol, for example a letter, number or punctuation mark.
String	A number of characters together, usually to form a word or sentence.
Casting	The process of forcing a value into a certain format, for example forcing a telephone number to be stored as a string and not an integer.
String Manipulation	A broad term for a number of ways to change or modify strings.
Concatenation	The process of joining two or more strings together.
Slicing	The process of splitting a string into two or more parts.
File Handling	Ways in which computer code can interact with files on the computer's disk, such as opening them or writing data to them.
Record	A single row or related information in a database table
Table	A group of records and fields stored together in a database.

Field	A part of a record in the database which stores specific information such as a name or address.
SQL	Structured Query Language. A programming language designed to be used with databases.
Array	A set of data items stored together with a single name and accessed by a program.
Sub Program	A smaller part of the program which performs a specific task.
Function	A sub program which performs a specific task and always returns a specific value when run.
Procedure	A sub program which performs a specific task but does not always return a value.
Random Number	A number generate by computer at random.

2.2.1 Programming Fundamentals

2.2.1 Programming fundamentals

- ☐ The use of variables, constants, operators, inputs, outputs and assignments
- ☐ The use of the three basic programming constructs used to control the flow of a program:
  - Sequence
  - Selection
  - Iteration (count- and condition-controlled loops)
- ☐ The common arithmetic operators
- ☐ The common Boolean operators AND, OR and NOT

Required

- ✓ Practical use of the techniques in a high-level language within the classroom
- ✓ Understanding of each technique
- ✓ Recognise and use the following operators:

Comparison operators		Arithmetic operators	
==	Equal to	+	Addition
!=	Not equal to	–	Subtraction
<	Less than	*	Multiplication
<=	Less than or equal to	/	Division
>	Greater than	MOD	Modulo
>=	Greater than or equal to	DIV	Quotient
		^	Exponentiation (to the power)

Concept	Meaning (student-friendly)	Code Example
Variable	A temporary value that can change during a program. Like a box holding data.	`score = 0` `lives = 3`
Updating a variable	Reassigning a new value to the same variable.	`score = score + 1` This is known as increment
Using in expressions	Variables can be used in calculations.	`myNumber = 5` `print(myNumber * 3)`
Constant	A value that must **not change** once set.	`const pi = 3.14`
Constant rule	Constants **cannot be reassigned**.	`pi = 4` ❌
Copying a constant	A variable copied from a constant can still change.	`my_pi = pi` `my_pi = 4`
Input	Gets data from the user (as a string).	`name = input("Enter name")`
Output	Displays data to the user.	`print(name)`
Input + output	Common exam pattern	`age = input("Age?")` `print(age)`

Operators

Operator type	What it is used for	Example (OCR style)	Operators	Common exam mistakes
Arithmetic operators	Perform calculations on numbers (maths).	total = 7 + 3	+ – * / DIV MOD ^	Using / instead of DIV when an integer answer is required; forgetting MOD gives the remainder; mixing strings and numbers without conversion.
Relational (comparison) operators	Compare values and return TRUE or FALSE.	age >= 18	== != < > <= >=	Using = instead of ==; writing assignment instead of comparison; forgetting comparisons return TRUE/FALSE, not numbers.
Boolean (logical) operators	Combine or change TRUE / FALSE values.	hasTicket AND isAdult	AND OR NOT	Using AND when OR is needed; forgetting NOT reverses the condition; missing brackets when combining conditions.

Arithmetic → does the maths
Relational → asks a question
Boolean → combines answers

Programming Constructs

Construct	Definition	Example	Example (OCR-style pseudocode)
Sequence	Instructions are carried out **one after another** in order, with no decisions or loops.	a = 2 b = 3 sum = a + b print(sum)	a = 2 b = 3 sum = a + b
Selection	The program **chooses** between **different paths** depending on a condition.		age = input("Enter age") age = int(age) **if age >= 18 then** print("Adult ticket")
Iteration	A section of code is **repeated** either a **set number** of times or while a condition is true.	for lessonNumber = 1 to 5 print("Lesson", lessonNumber)	**for lessonNumber = 1 to 5** print("Lesson", lessonNumber) **next lessonNumber**



Student memory tip

- ✓ **Sequence** → do this, then this
- ✓ **Selection** → if something happens, choose
- ✓ **Iteration** → repeat the same steps

Student memory tip

- ✓ **Sequence** → do this, then this
- ✓ **Selection** → if something happens, choose
- ✓ **Iteration** → repeat the same steps

OCR Exam Reference Language – Selection Comparison

IF / ELSEIF / ELSE

Best for: When you might need **ranges** or more flexible conditions

Key idea: Checks each condition in order until one matches

```
fruit = input('what is your favourite fruit?")
if fruit == "apple" then
    print("apples are red")
elseif fruit == "banana" then
    print("bananas are yellow")
elseif fruit == "blueberry" then
    print("blueberries are blue")
else
    print("I don't know what colour that is!"
endif
```

OR

SWITCH / CASE / DEFAULT

Best for: When matching one variable against a list of **exact values**

Key idea: Cleaner version of multiple "**elseif**" checks

```
fruit = input("what is your favourite fruit?")
switch fruit:
    case "apple":
        print "apples are red"
    case "banana":
        print "bananas are yellow"
    case "blueberry":
        print "blueberries are blue"
default:
        print "I don't know what
        colour that is!"
endswitch
```



Iteration (Loops) – OCR Exam Reference Language



Summary Table

Loop type	What it does	Key feature	ERL example

FOR (count-controlled)	Repeats code a set number of times	Runs from a start value to an end value (inclusive)	`for i = 1 to 10`
			`  print(i)`
			`  next i`

FOR with step	Repeats code using a **step value**	Can count up or down	`for i = 0 to 10 step 2` `  print(i)`

FOR countdown	Counts backwards	Uses a negative step	`for i = 5 to 0 step -1`
			`  print(i)`
			`  print("Blastoff!")`

WHILE (condition-controlled)	Repeats while a condition is **true**	May run zero times	`x = ""`
			`while x != "parrot"`
			`  x = input("password : ")`
			`  print "you guessed", xx`
			`endwhile`
			`  print "correct!"`
			`print("correct!")`

DO ... UNTIL (condition-controlled)	Repeats until a condition is **true**	Always runs at least once	

Key exam differences to remember

- ✓ **FOR** loops → use when you know **how many times**
- ✓ **WHILE** loops → check condition **before** running
- ✓ **DO...UNTIL** loops → check condition **after** running (runs once minimum)
- ✓ FOR loop bounds are **inclusive**
- ✗ **Do not change the loop variable** ('*i*') inside a FOR loop ✗

Practice coding using the examples below.
Variables Operators Selection Iteration Sequence

http://bit.ly/3L5kfwV

"You don't get good at coding by watching, you get good by practising." – Sister Teresa

2.2.1 Exam Questions on Programming Fundamentals

Question	Tip to answer (student)
1. Which statement is an example of **output**? (Tick one) ☐ input("Name?") ☐ print("Hello") ☐ age = 14 ☐ if age > 10 then (1) print("Hello") [1]	Output = shows something.
2. Which statement is an example of **input**? (Tick one) ☐ print("Enter age") ☐ age = input("Enter age") ☐ age = 12 ☐ age > 10 (1) age = input("Enter age") [1]	Input = user types.
3. State what is meant by a **variable**. (1) A named memory location that stores a value. [1]	"Stores data" in memory.
4. Explain why programs use **variables**. (2) Variables store data while the program runs.[1] The stored value can be used again or changed later.[2]	Store + reuse/change.
5. State what is meant by a **constant**. (1) A named value that cannot change while the program runs. [1]	Like a variable, but fixed.
6. Give one reason why a programmer might use a **constant**. (2) It prevents accidental changes to a fixed value (e.g. VAT).[1] If the value changes in future, you update it once.[2]	"Same value used lots" / easy update.
7. Which statement shows **assignment**? (Tick one) ☐ age == 10 ☐ age = 10 ☐ age >= 10 ☐ age != 10 (1) age = 10 [1]	Assignment uses one equals.
8. Explain the difference between **=** and **==** in many languages. (2) = stores a value in a variable (assignment).[1] == checks if two values are equal (comparison).[2]	Assign vs compare.
9. A program asks for a number and then prints double the number. Write an algorithm using input and output. (3) num = input("Enter a number")[1] answer = num * 2[2] print(answer)[3]	Input → calculate → print.
10. Identify the programming construct used: "instructions run one after another in order". (1) Sequencing [1]	That's one at a time.

Question	Tip to answer (student)
11. Identify the programming construct used: "the program chooses between two paths based on a condition". (1) Selection [1]	That's a decision.
12. Identify the programming construct used: "the program repeats a block of code". (1) Iteration [1]	That's repeating.

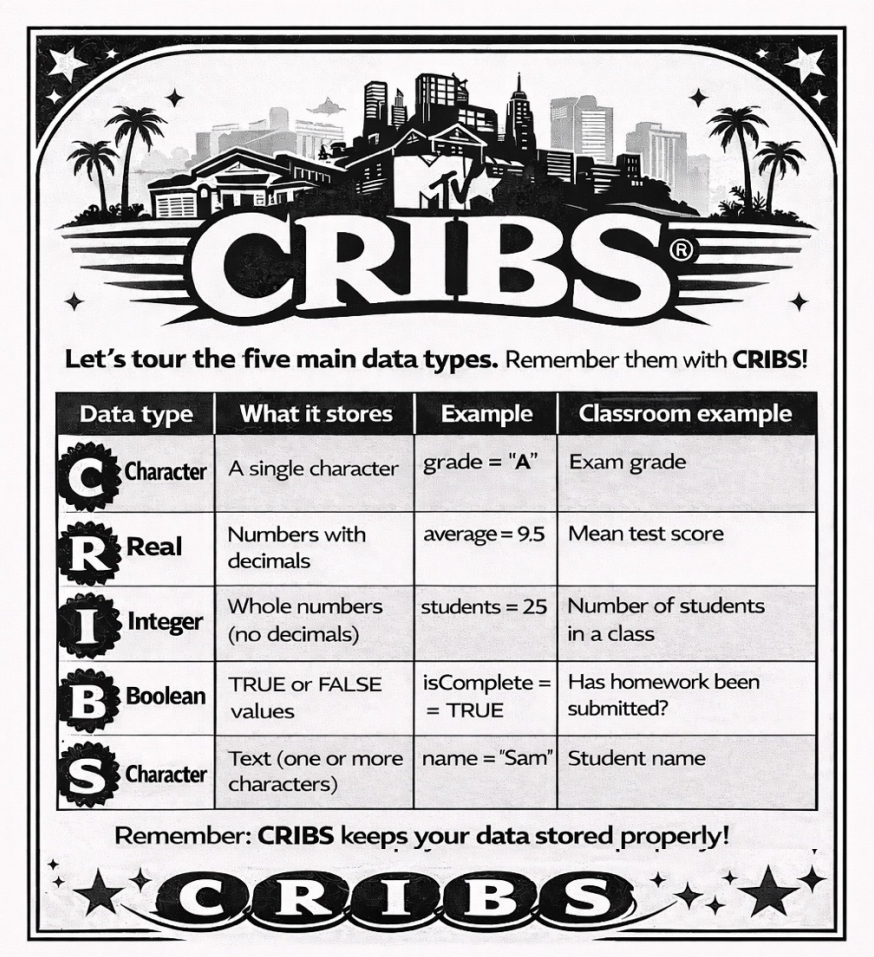
13. A program checks if a user is 13 or over and replies TRUE or FALSE. Complete the missing parts. (3)
`age = input("Age?")`
`if age ____ 13 then`
`  print("_____")`
`Else`
`  print("_____")`
`endif`
>=[1] True.[2] False
	Use >= or >?

14. A student writes: `if age >= 13 AND <= 18` Explain what is wrong and correct it. (3) The second condition is not a full comparison.[1] The variable must be written again.[2] Correct: age >= 13 AND age <= 18.[3]	Must repeat the variable.
15. Describe the difference between a **count-controlled** loop and a **condition-controlled** loop. (4) A count-controlled loop repeats a set number of times (e.g. FOR).[1] It uses a counter.[2] A condition-controlled loop repeats while/until a condition is met (e.g. WHILE / REPEAT UNTIL).[3] The condition decides when the loop stops.[4]	2 points each.
16. A loop runs while `x < 5`. x starts at 0 and increases by 1 each time. How many times will "OK" print? (2) It prints 5 times.[1] Values of x are 0,1,2,3,4.[2]	Count 0,1,2,3,4.
17. Complete the trace table for the algorithm. (4) \n\n01 total = 0 \n02 for i = 1 to 3 \n03 total = total + i \n04 next i \n05 print(total) total changes: 0 → 1 → 3 → 6[1] i values: 1,2,3[2] Output is 6[3] No extra changes.[4]	Track i and total.
18. Write an algorithm that keeps asking for a password until the user types `"letmein"`. (6) pass = input("Password?")[1] while pass != "letmein"[2] print("Try again")[3] pass = input("Password?")[4] endwhile[5] print("Access granted")[6]	Needs loop + input + compare.
19. Explain one use of the operator **MOD** in programs. (2) MOD gives the remainder after division.[1] It can check if a number is even (num MOD 2 == 0).[2]	Remainder / even-odd.

2.2.2 Data Types

2.2.2 Data types	
☐ The use of data types: ○ Integer ○ Real ○ Boolean ○ Character and string ○ Casting	**Required** ✓ Practical use of the data types in a high-level language within the classroom ✓ Ability to choose suitable data types for data in a given scenario ✓ Understand that data types may be temporarily changed through casting, and where this may be useful

A **data type** tells the computer **what kind of data** is being stored and **how it can be used**.



Scenario	Best data type	Why	Common misconception
Number of students	Integer	You can't have part of a student	Using **Real** because "numbers can have decimals"
Test average	Real	Can include decimals	Using **Integer** and losing the decimal part
Homework handed in	Boolean	Only TRUE or FALSE	Using **String** like `"yes"` / `"no"`
GCSE grade	Character	Single letter	Using **String** instead of a single character
Student name	String	Text	Using **Character**, which only stores one letter

Casting means **temporarily changing** a value from one data type to another.
This is often needed because `input` usually returns a s**tring** nd maths needs **numbers**

Casting	Example	Why it's needed
String → Integer	`age = int(age)`	To do maths with input
Integer → String	`print("Score: " + str(score))`	To display numbers with text
Integer → Real	`average = real(total) / real(count)`	To avoid whole-number division

If the question asks you to **choose a data type**, explain **why your choice fits the data** — that's where the marks are.

A school computer system stores the following information: the number of students in a class
the average test score for the class, whether homework has been submitted and a student's exam grade
For each item, **state a suitable data type** and **justify your choice**. (4 marks)

Model answer (4 marks)

- The number of students should be stored as an **Integer** because it is a whole number and cannot have decimals.[1]
- The average test score should be stored as a **Real** because it may include decimal values.[2]
- Whether homework has been submitted should be stored as a **Boolean** because it only has two possible values: TRUE or FALSE.[3]
- A student's exam grade should be stored as a **Character** because it is a single letter such as A or B.[4]

Misconception	Why students think this	Why it is wrong
Using **Real** instead of **Integer**	"All numbers should be Real."	Whole counts (e.g. number of students) cannot have decimal values.
Using **String** instead of **Boolean**	Using `"yes"` / `"no"` feels easier to understand.	Boolean is designed for two-state answers (TRUE / FALSE) and is clearer and more efficient.
Using **String** instead of **Character**	GCSE grades are text, so must be a String.	A single letter (e.g. A) is best stored as a Character.
Using **Integer** for averages	Averages look like normal numbers.	Averages often include decimal values, so must be stored as Real.
Forgetting to cast after input	Input can be used directly in calculations.	`inpu

2.2.2 Exam Questions on Data Types

Question	Tip to answer (student)
1. Which data type is best for storing **True/False**? (Tick one) (1) ☐ Integer ☐ Boolean ☐ Character ☐ String Boolean [1]	True/False = Boolean.
2. State what is meant by a **data type**. (1) A data type describes what kind of value is stored (e.g. integer, string). [1]	"Type of data" stored.
3. Give the most suitable data type for a person's **age**. (1) Integer [1]	Age is a whole number.
4. Give the most suitable data type for a person's **first name**. (1) String [1]	Names are text.
5. Give the most suitable data type for **grade = 'A'**. (1) Character [1]	Single letter.
6. Give the most suitable data type for **height = 1.72**. (1) Real / Float [1]	Has decimals.
7. Explain why a program must know the data type of a variable. (2) It helps the computer store the data correctly (correct memory size/format).[1] It also allows the computer to process it correctly (e.g. maths on numbers, not strings).[2]	Storage size + what you can do with it.
8. A student stores `"2"` in a variable. Explain why `"2" + "2"` gives `"22"` in some languages. (2) The values are strings, not numbers.[1] The + joins text together, so "2" + "2" becomes "22".[2]	String concatenation.
9. A program asks for an age using input(). Explain why the age may need **casting** before doing calculations. (2) input() usually returns text (a string).[1] Casting changes it to an integer so maths can be done correctly.[2]	Input often comes in as a string.
10. State what **casting** means. (1) Converting data from one type to another (e.g. string to integer). [1]	Converting types.
11. A variable `age` currently stores the string `"14"`. Write one line of code/pseudocode to cast it to an integer. (1) age = int(age) [1]	Use int().
12. A variable `score` stores the integer `7`. Write one line of code/pseudocode to cast it to a string. (1) score = str(score) [1]	Use str().
13. Tick (✓) the correct statement: An integer is… (1) ☐ a whole number ☐ text ☐ one character ☐ True/False a whole number [1]	Whole number.
14. Tick (✓) the correct statement: A character is… (1) ☐ multiple words	Single character only.

Question | **Tip to answer (student)**

☐ one symbol/letter/digit
☐ a decimal number
☐ True/False

one symbol/letter/digit [1]

15. Explain the difference between a **string** and a **character**. (2) | One vs many.

A character is one single symbol (e.g. 'A').[1]
A string is a sequence of characters (e.g. "Amelia").[2]

16. A program stores whether a user is logged in.
Give a suitable variable name and data type. (2) | Name + Boolean.

Example: isLoggedIn[1] as Boolean[2]

17. A student writes: `total = "10" + 5` and gets an error. Explain why and show one fix using casting. (3) | Mixed types. Fix by casting string→int or 5→string.

"10" is a string but 5 is an integer, so they can't be combined as numbers/text safely.[1] Cast "10" to int:
total = int("10") + 5[2] (or cast 5 to string for joining text).[3]

18. A shop stores prices like 2.50 and 19.99.
Explain why **integer** is not suitable and give the correct type. (2) | Needs decimals.

Prices can include decimal values, so integers would lose the fractional part.[1] Use Real/Float.[2]

19. Complete the table by choosing the most appropriate data type. (4) | Think: text / whole / boolean / char.

Variable	Data Type
Username	String
Number of Lives	Integer
Game Over (True/False)	Boolean
Menu Choice (Single Letter)	Character

20. A program asks for two numbers as input and outputs their total. Complete the missing lines. (4) | Need int() both times.

```
\n\nnum1 = input("Enter number 1") \nnum2 = input("Enter number 2") \nnum1 = _____(num1) \nnum2 = _____(num2) \nprint(num1 + num2)
```

int[1] int[2] Correct casting of both inputs[3] Correct total output as number[4]

Explain one reason why Boolean only needs **1 bit**. (2) | Only two states.

Boolean has only two values (True/False).[1] Two states can be stored using one binary digit (0/1).[2]

2.2.3 Additional Programming Techniques

2.2.3 Additional programming techniques	
☐ The use of basic string manipulation ☐ The use of basic file handling operations: ○ Open ○ Read ○ Write ○ Close ☐ The use of records to store data ☐ The use of SQL to search for data ☐ The use of arrays (or equivalent) when solving problems, including both one-dimensional (1D) and two-dimensional arrays (2D) ☐ How to use sub programs (functions and procedures) to produce structured code ☐ Random number generation	**Required** ✓ Practical use of the additional programming techniques in a high-level language within the classroom ✓ Ability to manipulate strings, including: ▪ Concatenation ▪ Slicing ✓ Arrays as fixed length or static structures ✓ Use of 2D arrays to emulate database tables of a collection of fields, and records ✓ The use of functions ✓ The use of procedures ✓ Where to use functions and procedures effectively ✓ The use of the following within functions and procedures: ▪ local variables/constants ▪ global variables/constants ▪ arrays (passing and returning) ✓ SQL commands: ▪ SELECT ▪ FROM ▪ WHERE ✓ Be able to create and use random numbers in a program

2.2.3 String Handling

A **string** is text made up of characters, for example message = “You are amazing!”. String manipulation is **changing or working with text in a program**, such as joining, shortening, or changing letters.

message	Y	o	u	(space)	a	r	e	(space)	a	m	a	z	i	n	g	!
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Indexing is how a computer counts positions in a string.
The computer starts counting from 0, not 1.Using the message: `"You are amazing!"`
The letter Y is at index 0
The letter o is at index 1
The letter u is at index 2
The space after *You* is at index 3
The last character ! is at index 15
So when we say index, we mean the position of a character in the text, starting from 0.

Feature	What it does	Example code	Result
String	Stores text	subject = "ComputerScience"	`"ComputerScience"`
Length `.length`	Counts characters in a string	subject.length	`15`
Substring `.substring(x, i)`	Takes part of a string (from index x, i characters)	subject.substring(3, 5)	`"puter"`
Left `.left(i)`	Takes first i characters	subject.left(4)	`"Comp"`
Right `.right(i)`	Takes last i characters	subject.right(3)	`"nce"`
Concatenation +	Joins strings together	"Hello " + name	`"Hello Sam"`
Uppercase `.upper`	Converts to uppercase	subject.upper	`"COMPUTERSCIENCE"`
Lowercase `.lower`	Converts to lowercase	subject.lower	`"computerscience"`
ASCII `ASC()`	Converts character to number (	ASC('A') ASC('B') ASC('C')	65 66 67
Character `CHR()`	Converts number to character	CHR(97) CHR(98) CHR(99)	'a' 'b' 'c'

2.2.3 One Dimensional Arrays

Strings are sequences of characters and work like one-dimensional arrays, which helps programmers understand indexing. One-dimensional arrays store lists of data under one variable name. A 1D array could contain **c**haracters, ***r***eals, **i**nts, **b**ooleans or **s**trings.

Each item in the 1D array is referenced by an index, starting at **zero**.

1D Array Question Scores = [65, 72, 81, 90, 58]	Scores = [65, 72, 81, 90, 58]
Output the first score.	Print (Scores[0])
Output the last score in the array.	Print Scores([4])
Output all values in the array (one per line).	FOR i = 0 TO 4 Print (Scores[i]) NEXT i
Find the total (sum) of the scores in the array.	Total =0 FOR i = 0 TO 4 Total =Total + Scores[i] NEXT i Print (Total)
Find the average of the 5 scores in the array.	Total = 0 FOR i = 0 TO 4 Total = Total + Scores[i] NEXT i Average = Total / 5 Print (Average)
Count how many values in the array are greater than 50.	Count = FOR i <- 0 TO 5 IF Scores[i] > 50 THEN Count <- Count + 1 ENDIF NEXT i Print (Count)
Find the highest number in the array.	Max = Numbers[0] FOR i <- 1 TO 5 IF Numbers[i] > Max THEN Max = Numbers[i] ENDIF NEXT i Print (Max)

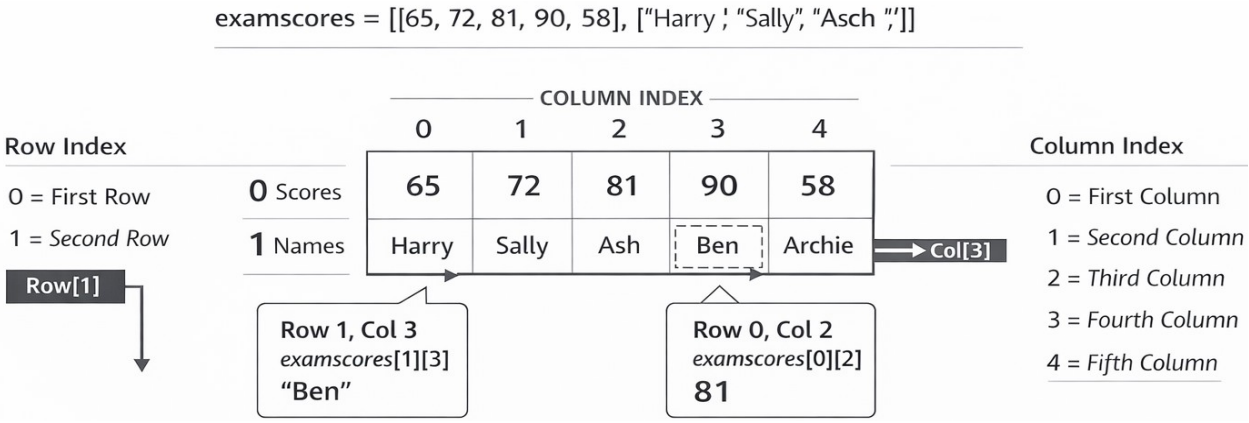
Swap two elements (swap index 1 and index 3), then output the array.	Temp = A[1]
	A[1] = A[3]
	A[3] = Temp //Swapped them – Think bubble!
	FOR i = 0 TO 4
	  Print A[i]
	NEXT i

Replace every value in the array with itself doubled, then output the array.	FOR i = 0 TO 4 Numbers[i] =Numbers[i] * 2 NEXT i FOR i = 0 TO 4

	Print Numbers[i] NEXT i

2.2.3 Two Dimensional Arrays

Two-dimensional arrays store data in rows and columns like a table.



2D Array Question (Tic-Tac-Toe)	Code
2D Array Setup (empty board)	board = [["", "", ""], ["", "", ""], ["", "", ""]]

Put an **X** in the centre square.
`| | | |`
`| | X | |`
`| | | |`
	board[1][1] = "X"

Put an **O** in the top-left square.
`| O | | |`
`| | | |`
`| | | |`
	board[0][0] = "O"

Print the value in the bottom-right square.
`| O | | |`
`| | O | |`
`| | | X |`
	print(board[2][2])

Print the whole board (all 9 squares), one per line.	FOR r = 0 TO 2 FOR c = 0 TO 2 print(board[r][c]) NEXT c NEXT r
Check if **X** has won across the **middle row**. Print TRUE/FALSE.	IF board[1][0] = "X" AND board[1][1] = "X" AND board[1][2] = "X" THEN print(TRUE) ELSE print(FALSE) ENDIF

Check if **O** has won down the **first column**. Print TRUE/FALSE.	IF board[0][0] = "O" AND board[1][0] = "O" AND board[2][0] = "O" THEN print(TRUE) ELSE print(FALSE) ENDIF

2.2.3 File Handling

File handling is **opening, reading from, writing to, and closing files** so a program can save and use data

Action	What it does (simple)	Example code	Common mistakes
Open file	Opens a file so it can be used	myFile = open("sample.txt")	Forgetting to store the file in a variable
Read line	Reads the next line from the file	myFile.readLine()	Expecting it to read the whole file at once
Write line	Writes a new line to the **end** of the file	myFile.writeLine("Add new line")	Thinking it overwrites the file instead of adding
End of file check	Checks if there are no lines left to read	myFile.endOfFile()	Forgetting `NOT` in the while condition
Read whole file	Reads every line until the end	while NOT myFile.endOfFile() print(myFile.readLine())endwhile	Calling `readLine()` twice inside the loop
Close file	Closes the file when finished	myFile.close()	Forgetting to close the file after use
Create new file	Creates a new text file	newFile("myText.txt")	Forgetting the file must still be opened before writing

4 Mark, typical question

A text file is used to store pilot's codes and date of birth.

The text file is named pilots.txt. The contents of some of this file is shown

```
JP670,17/05/1986
CC200,31/06/1980
SM720,01/04/1976
GC730,06/06/1978
```

Design an algorithm to:

- Ask the user to input a pilot code and a date of birth
- Write these inputs to the pilots.txt text file

1 mark Open pilots.txt

1 mark Ask the user for two **inputs** and **store in variables**

```
1 dronepilotData = open("pilots.txt")
2 pilotCode = input("enter code")
3 dob = input("enter date of birth")
4 dronepilotData.writeLine(pilotCode, dob)
5 dronepilotData.close()
```

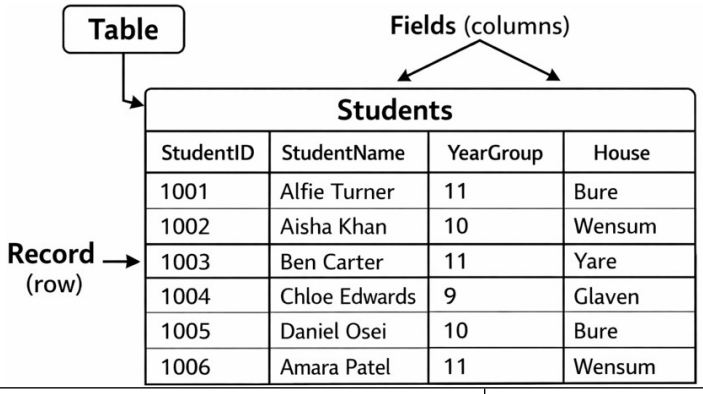
1 mark Write both inputs to opened text file

1 mark Close text file

2.2.3 SQL

SQL is used to **query a database** (ask questions and get data back) from tables.
SELECT – chooses which **fields (columns)** you want to see in the results.
FROM – tells the database which **table** the data comes from.
WHERE – filters the **records (rows)** so only those that meet a condition are shown.

- **=** means **equal to**, so records with that value are included
- **<>** means **not equal to**, so records with that value are excluded.
- **AND** makes sure **all conditions** must be true for a record to be returned.
- **OR** returns a record if **at least one condition** is true.
- **<>** means **not equal to**, so records with that value are excluded.
- **LIKE** – searches for text that **matches a pattern**, such as names starting with a letter.
- ***** – means **all fields** in the table



Students

StudentID	StudentName	YearGroup	House
1001	Alfie Turner	11	Bure
1002	Aisha Khan	10	Wensum
1003	Ben Carter	11	Yare
1004	Chloe Edwards	9	Glaven
1005	Daniel Osei	10	Bure
1006	Amara Patel	11	Wensum

SQL query	Records returned
SELECT * FROM Students;	1001, Alfie Turner, 11, Bure 1002, Aisha Khan, 10, Wensum 1003, Ben Carter, 11, Yare 1004, Chloe Edwards, 9, Glaven 1005, Daniel Osei, 10, Bure 1006, Amara Patel, 11, Wensum
SELECT StudentName, House FROM Students;	Alfie Turner, Bure Aisha Khan, Wensum Ben Carter, Yare Chloe Edwards, Glaven Daniel Osei, Bure Amara Patel, Wensum
SELECT StudentID, StudentName FROM Students WHERE YearGroup = 11;	1001, Alfie Turner 1003, Ben Carter 1006, Amara Patel
SELECT StudentName, YearGroup FROM Students WHERE House = 'Bure';	Alfie Turner, 11 Daniel Osei, 10
SELECT StudentID, StudentName FROM Students WHERE YearGroup = 10 AND House = 'Bure';	1005, Daniel Osei
SELECT StudentName, House FROM Students WHERE House = 'Wensum' OR House = 'Yare';	Aisha Khan, Wensum Ben Carter, Yare Amara Patel, Wensum
SELECT StudentID, StudentName, House FROM Students WHERE House <> 'Bure';	1002, Aisha Khan, Wensum 1003, Ben Carter, Yare 1004, Chloe Edwards, Glaven 1006, Amara Patel, 11, Wensum
SELECT StudentID, StudentName FROM Students WHERE StudentName LIKE 'A%';	1001, Alfie Turner 1002, Aisha Khan 1006, Amara Patel

Write SQL (3 mark question)

The log is stored in a database table called `events`. The current contents of `events` is shown:.

Date	SensorID	SensorType	Length
05/02/2023	WS2	Window	38
05/02/2023	MS1	Motion	2
06/02/2023	DS3	Door	1
06/02/2023	MS2	Motion	3
06/02/2023	MS1	Motion	2
07/02/2023	WS1	Window	24
07/02/2023	DS1	Door	1

Write an SQL statement to display the sensor IDs of the door sensors that have been triggered for more than 20 seconds.

1 mark SELECT SensorID or you could use WILDCARD **SELECT ***

1 mark **FROM** events

1 mark **WHERE** Length > 20 **AND** sensorType = "Door"

This would return no records as all Doors are open for one second. Well done OCR!

2.2.3 2D arrays to emulate SQL

Let's write the SQL to output all windows that are open for >30 seconds. Include the Date and SensorID.

SQL	Code
SELECT Date, SensorID FROM events WHERE SensorType = 'Window' AND Length > 30; This would return 05/02/2023 Window	eventfile = open("events.csv") line = readline() // skip header row while NOT eventfile.endOfFile() line = readline() Date = parts[0] SensorID = parts[1] SensorType = parts[2] Length = parts[3] if SensorType = "Window" AND Length > 30 then print(Date + " " + SensorID) endif endwhile eventfile.close()

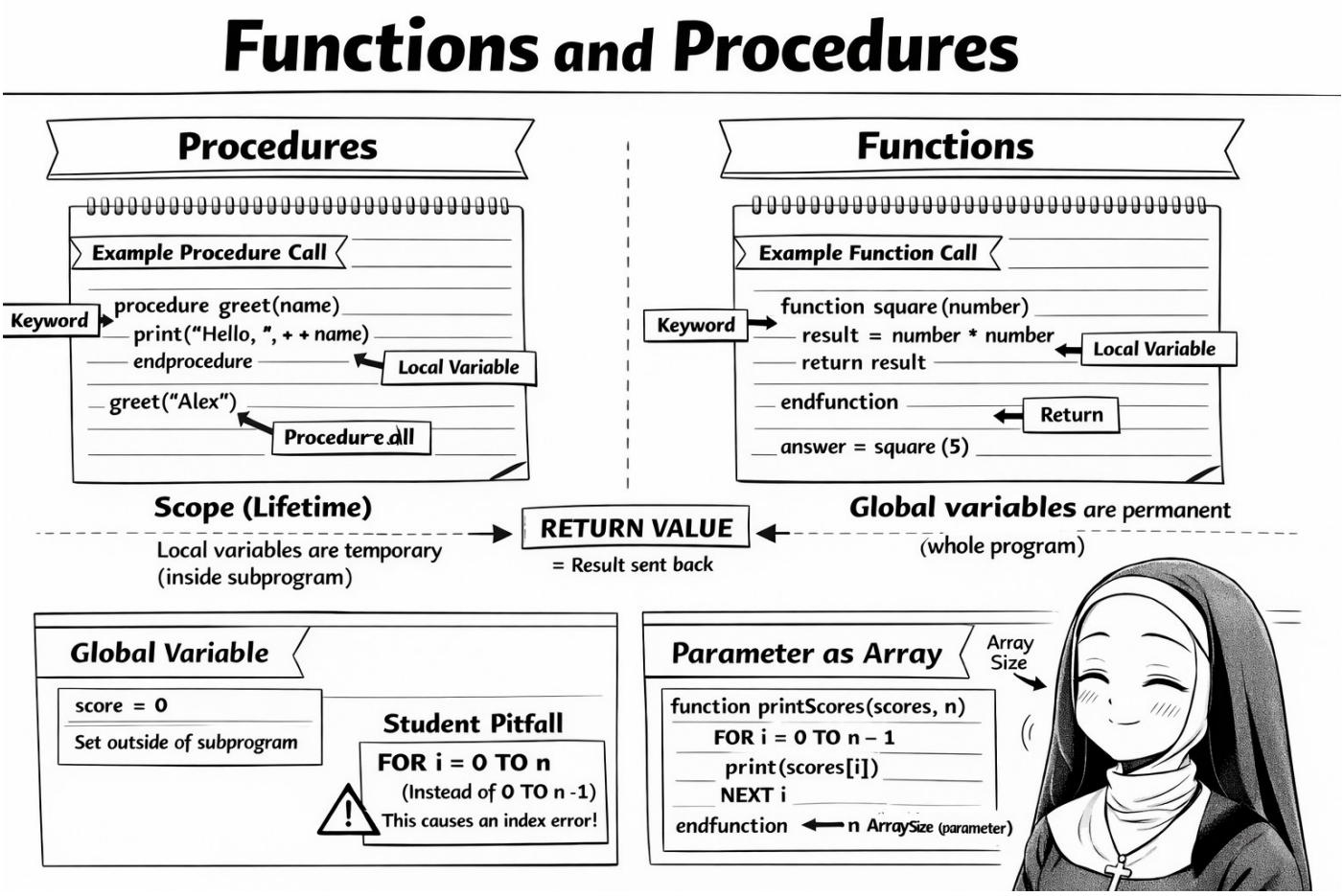
Iterate through records → check field values → print matching records.

2.2.3 Subprograms

A subprogram is a section of code that carries out a task and can be reused. There are two different types of subprograms – procedures and functions. **Procedures** are used to perform tasks such as **displaying output**, while **functions** are used to calculate and **return values** that can be **reused** elsewhere in a program.

👉 **Procedures DO** 👉 **Functions RETURN**

Using subprograms helps: reduce repeated code, make programs easier to read and make programs easier to fix and test (think coding teams).



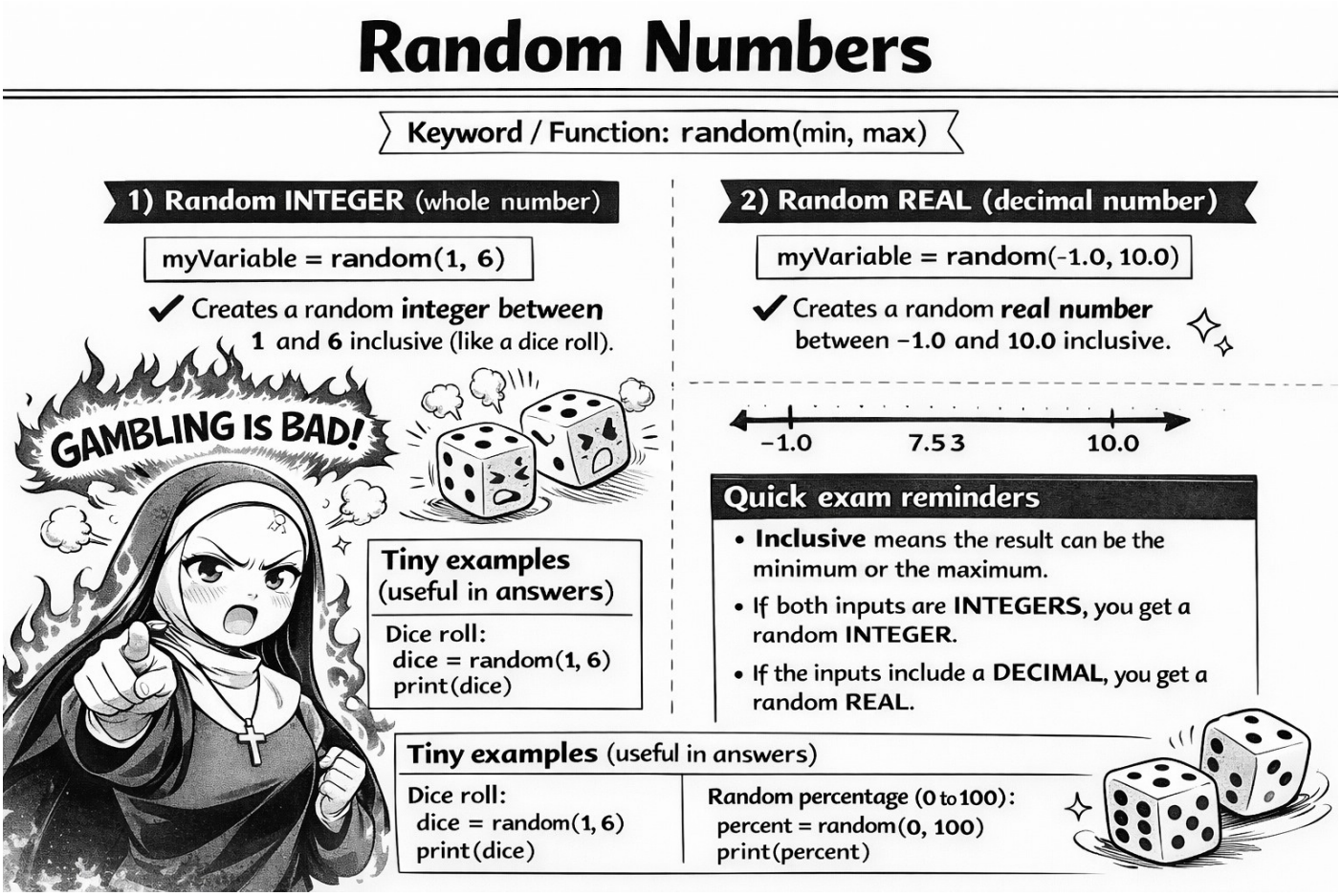
Procedure (code examples + call)	Function (code examples + call)
procedure agePass() print("You are old enough to ride") endprocedure //Main **agePass()**	function squared(number) return number ^2 endfunction //Main **result =squared(4)**
Procedure printName(name) print(name) endprocedure //Main **printName("Alex")**	function add(a, b) return a + b endfunction //Main **total = add(3, 5)**

procedure multiply(num1, num2)
  print(num1 * num2)
endprocedure
multiply(3, 4)
	function multiply(num1, num2)
	return num1 * num2 endfunction
	//Main
	result = multiply(3, 4)

procedure showScore (score)
  print(score)
endprocedure
//Main
currentscore =10
showScore(currentscore)
	function average(Score1, Score2, Score3)
	  return (Score1 + Score2 + Score3) / 3
	endfunction
	//Main
	Score1 =12
	Score2 =14
	Score3 = 16
	avg = average(Score1, Score2, Score3)

procedure printScores(scores, n)
  n = LENGTH(scores)
  FOR i = 0 TO n - 1
  print(scores[i])
  NEXT i
endprocedure
 	
// Main program
scores = [10, 3, 4, 5]
printScores(scores, 4)
	function updateScores(scores, n)
	  n = LENGTH(scores)
	  FOR i = 0 TO n - 1
	  scores[i] = scores[i] + 1
	  NEXT i
	  return scores
	Endfunction
	
	// Main program
	scores = [10, 3, 4, 5]
	scores = updateScores(scores, 4)

2.2.3 Random numbers



Practice writing out code on paper like in the exam – Sister Teresa

Apply functions and selection: Write a function deliveryCost(parcels) which returns parcels * 4. In the main algorithm store deliveryCost(6) in totalCost; if the returned value exceeds 20, output "Large order", otherwise output "Standard order". A function returns a value; the calling code decides what to do with it.

OCR ERL – combining file handling and casting: Open "scores.txt"; while NOT myFile.endOfFile(), read a line into score, convert it using int(score), and print only scores greater than 50. Close the file after the loop. Explain why casting is needed when comparing the input string with a number.

2.2.3 Exam Questions on Additional Programming Techniques

Question	Tip to answer (student)
1(a) What does `resources.length` return for `resources = "ComputerScienceUK"`? **(1)**	Length = number of characters.
`17`	
1(b) What is returned by `resources.left(4)` for `resources = "ComputerScienceUK"`? **(1)**	left(4) = first 4 characters.
`"Comp"`	
1(c) What is returned by `resources.right(3)` for `resources = "ComputerScienceUK"`? **(1)**	right(3) = last 3 characters.
`"eUK"`	
1(d) What does `ASC("B")` return? **(1)**	ASC = letter → number.
`66`	
1(e) What does `CHR(98)` return? **(1)**	CHR = number → character.
`"b"`	
1(f) Which one is a **valid** way to open a file for reading? **(1)** **A** `myFile = open("results.txt")` **B** `open("results.txt") = myFile` **C** `myFile.open("results.txt")` **D** `open myFile "results.txt"`	file handler on the left.
A	
2(a) State two reasons a program should close a file after using it. **(2)**	Two short points. Think "save properly" + "avoid errors".

Question	Tip to answer (student)
Stops data being lost / ensures writes are finished.[1]Prevents errors / frees the file so other programs can use it.[2]	
2(b) State two features of an **array (2)**	Fixed size + same data type.
Declared with a fixed length .[1]Stores items of the same data type.[2]	
3(a) Explain what **concatenation** is, using a string example. **(3)**	Say "join strings" + show example.
Concatenation means joining strings together.[1]It uses + to combine them.[2]Example: `print("Hello " + name)` outputs one longer string.[3]	
3(b) A program reads `temp = myFile.readLine()` and then checks `if int(temp) > 10`. Explain why **casting** is used here. **(3)**	"readLine gives text" → convert to number to compare.
readLine() returns a string (text).[1]The program needs a number to compare with 10.[2]int(temp)converts the string to an integer so the comparison works.[3]	
3(c) Explain the difference between a **procedure** and a **function**. **(3)**	Procedure = no return. Function = returns a value.
Both are subroutines (named blocks of code).[1]A procedure runs code but does not return a value.[2]A function returns a value back to the main program.[3]	
4(a) Write an SQL statement to show **ActID and Stage** for acts playing on **Saturday** in `TblMusic(ActID, Stage, Day, SetLength)`. **(4)**	Commas between fields. WHERE not IF. Quotes for Saturday.
`SELECT ActID, Stage`[1]`FROM TblMusic`[2]`WHERE Day = "Saturday"`[34]	
4(b) Write an algorithm that reads every line from `results.txt` and outputs the line **only if it exceeds 10**. Use OCR Exam Reference Language file handling. **(4)**	open → while NOT endOfFile → readLine → IF → print → close

```
myFile = open("results.txt")[1]
while NOT myFile.endOfFile()[2]
  temp = myFile.readLine()[3]
  if int(temp) > 10then[4]
   print(temp)[4]
    endif
endwhile
myFile.close()
```

4(c) An array `scores` stores 5 integers. Write an algorithm to output the **total** of all values in `scores`. **(4)**

total starts at 0, loop through indexes, add, then print.

```
total = 0[1]
for x = 0 to 4[2]
  total = total + scores[x][3]
next x
print(total)[4]
```

Question	Tip to answer (student)
4(d) Write a **procedure** called `showWelcome()` that outputs `Welcome to Community Arts Night`, then **call** it. **(4)**	Procedure = no return. Define it, then call it.

```
procedure showWelcome()[1]
  print("Welcome to Community Arts Night")[2]
endprocedure[3]
showWelcome()[4]
```

4(e) Write a **function** called `calculatePrice(tickets)` that returns the total cost at **£12 per ticket**. Then call it for 4 tickets and store the result in `totalPrice`. **(4)**	Function must RETURN (not print). Use the parameter.

```
function calculatePrice(tickets)[1]
  total = tickets * 12[2]
  return total[3]
endfunction
totalPrice = calculatePrice(4)[4]
```

6(a) A 2D array stores stage data like a table: `acts = [[1,"Platinum","Saturday",0.5],[2,"Hills","Saturday",1.5]]` Explain how a 2D array can **represent records and fields**. **(6)**	Row = record, column = field. Use an example with indices.

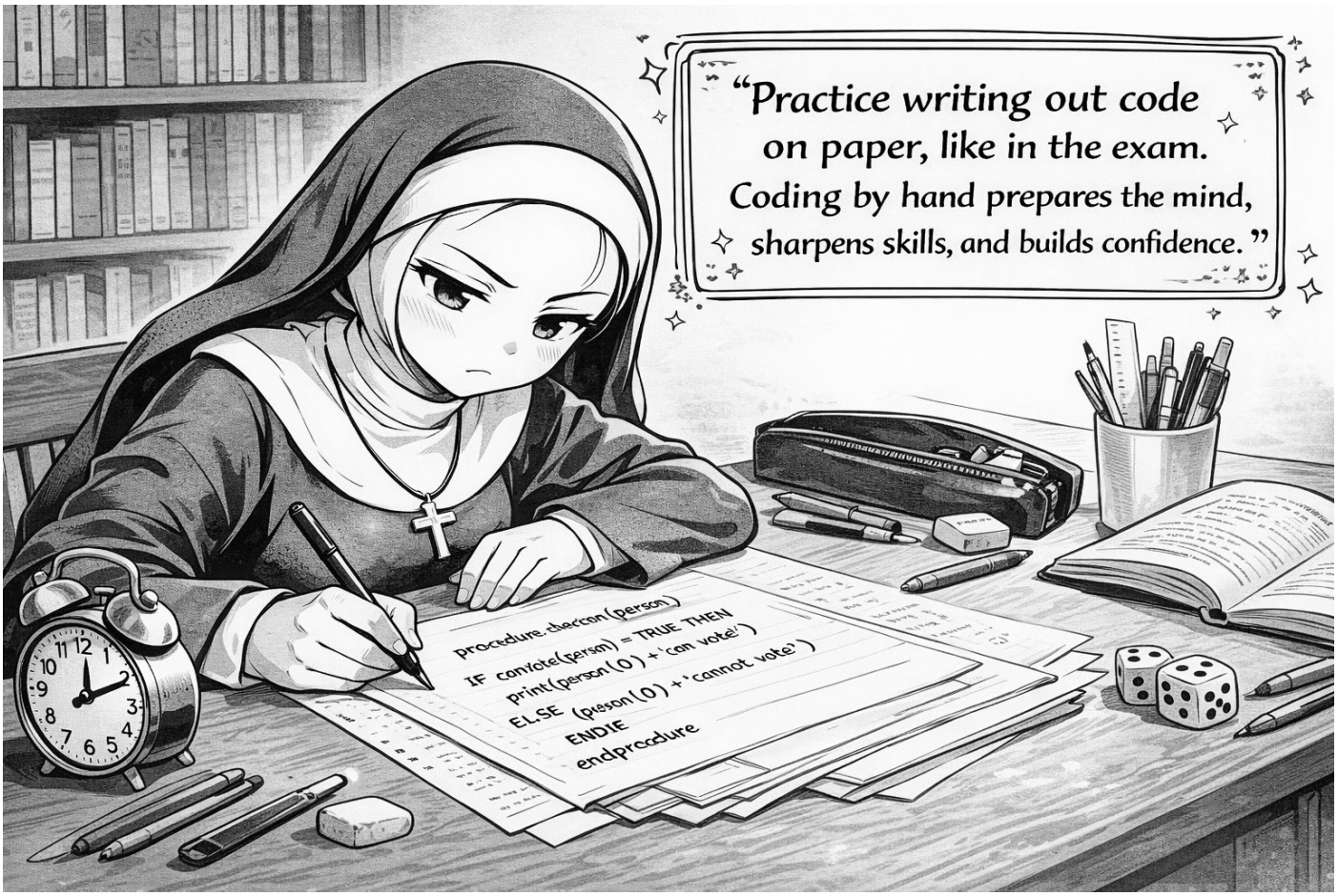
Each inner list is one row, so it represents one record.[1]Each position inside the inner list is one field (ActID, Stage, Day, SetLength).[2]acts[0] gives the first record.[3]acts[0][1] gives the Stage field from the first record ("Platinum").[4]This is like a database table with rows and columns.[5]It lets a program store and process many records in one structure.[6]

6(b) Library lending algorithm: There are **90** books available. A user can borrow **up to 10** books at a time. The program repeatedly asks how many books they want. If enough books remain, output **Books issued** and reduce `books`. Otherwise output **Not enough books**. The program stops when `books` is **0**. Write an algorithm in OCR Exam Reference Language style. **(6)**	Loop while books > 0. Check 1–10. Only subtract when lending succeeds.

```
books = 90
while books > 0[1]
num = int(input("How many books (1 to 10)?"))[2]
if num<1 OR num>10 then[3]
   print("Not accepted")[3]
else
   if books >= num then[4]
      print("Books issued")[5]
      books = books - num[6]
   else
      print("Not enough books")[5]
   endif
endif
endwhile
```

6(c) Write an algorithm that chooses a random whole number from **1 to 6** and outputs `You rolled:` followed by the number. **(6)**	Use `random(1,6)` then output message + value.

```
roll = random(1,6)[1]
if roll >= 1 AND roll <= 6 then[2]
 print("You rolled: " + roll)[3][4]
else
 print("Error")[5]
endif(A roll is produced each time the program runs.)[6]
```

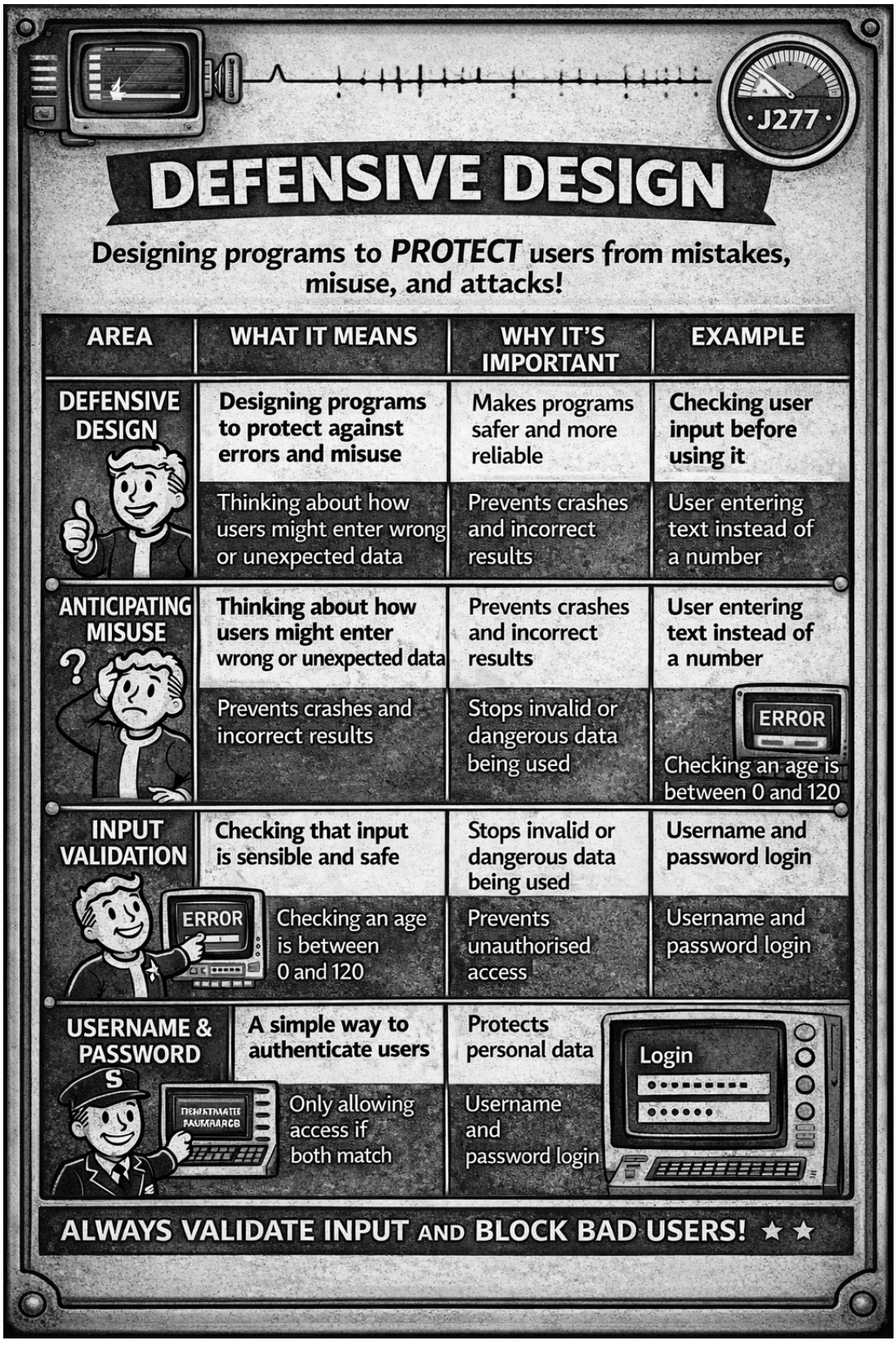

"Practice writing out code
on paper, like in the exam.
Coding by hand prepares the mind,
sharpens skills, and builds confidence."
procedure checkcan(person)
IF canvote(person) = TRUE THEN
print(person(0) + 'can vote')
ELSE (pieson(0) + 'cannot vote')
ENDIE
endprocedure

2.3 Producing Robust Programs

Keyword	Definition
Defensive Design	Designing and building computer code with the ability to keep it secure, easy to maintain and robust as a priority.
Authentication	A security mechanism which requires a password, code, fingerprint or other factor to gain access to all or part of a system.
Input Validation	The process of comparing data entered into the program to a set of rules to ensure it is valid and will not damage the system before processing it.
Maintainability	The ability to keep code updated, fix bugs and add new features easily.
Naming Conventions	An agreed way of naming parts of the program such as variables or sub programs such that they are easy to read and their purpose is clear.
Indentation	Inserting spaces at the start of a line or lines of code so to group them together and make the code easier to read.
Comment	Lines which do not form part of the code and are not executed but instead provide notes and explanations to those looking at the code.
Testing	The process of trialling a program in order to ensure it performs as expected.
Iterative Testing	Testing which is carried out whilst the program is being developed.
Final Testing	Testing which is carried out at the end of development just before the code is released.
Syntax Error	An error which breaks the rules of the programming language and stops the code from running.
Logic Error	An error which produces an unexpected output even though the code still runs.
Normal Data	Data which the program should accept without error.
Boundary Data	Data at the extremes of what the program should accept without error.
Invalid Data	Data of the correct type, but not valid for the program and which should be rejected.
Erroneous Data	Data which is the wrong type and should be rejected.

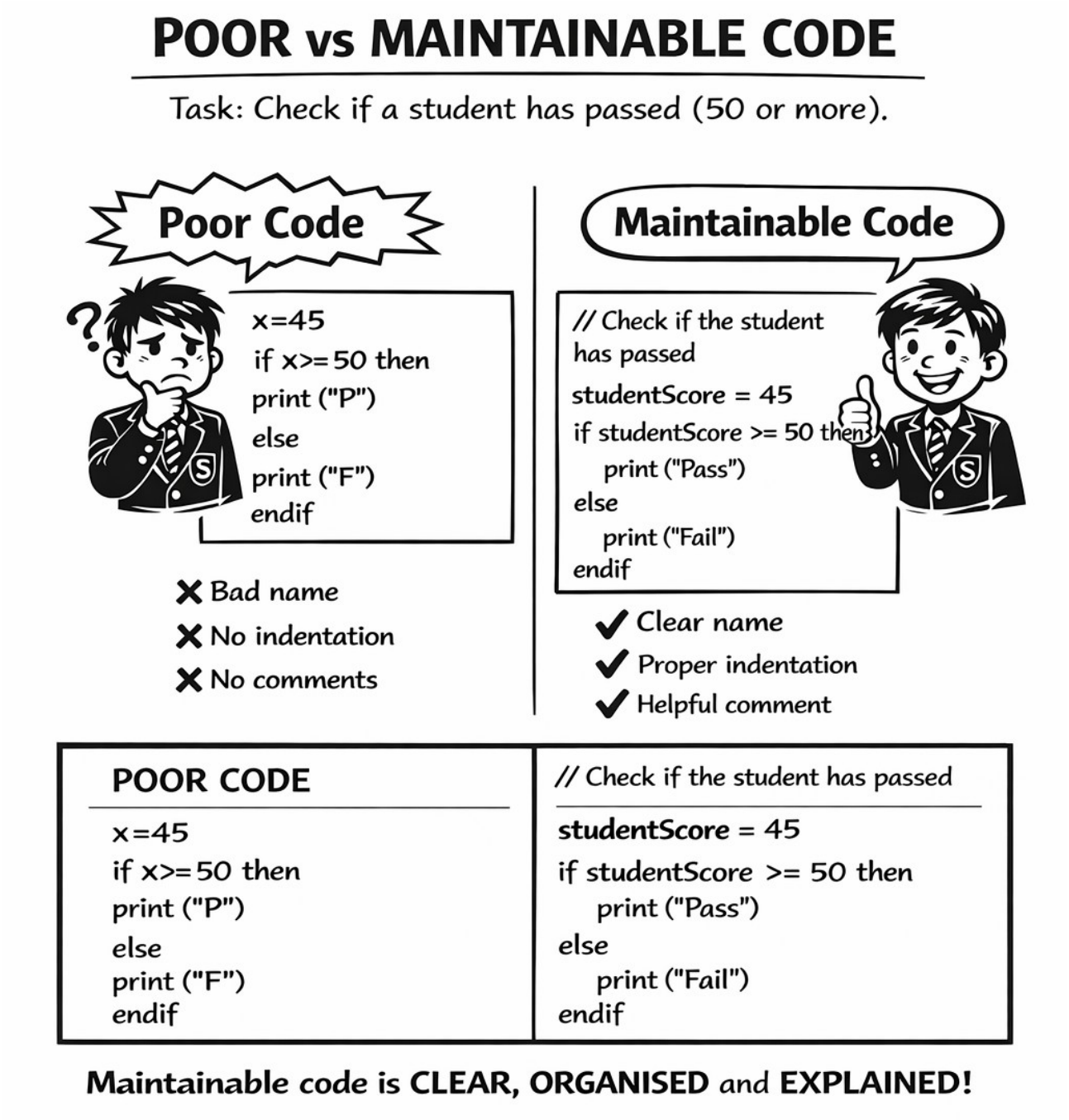
2.3.1 Defensive Design

2.3.1 Defensive design	
☐ Defensive design considerations: ○ Anticipating misuse ○ Authentication ☐ Input validation ☐ Maintainability: ○ Use of sub programs ○ Naming conventions ○ Indentation ○ Commenting	**Required** ✓ Understanding of the issues a programmer should consider to ensure that a program caters for all likely input values ✓ Understanding of how to deal with invalid data in a program ✓ Authentication to confirm the identity of a user ✓ Practical experience of designing input validation and simple authentication (e.g. username and password) ✓ Understand why commenting is useful and apply this appropriately



Basic	username = input("Enter username") password = input("Enter password") if username == "admin" AND password == "letmein" then print("Access granted") else print("Access denied") endif
Stops Brute force	correctUser = "admin" correctPass = "letmein" attempts = 0 maxAttempts = 3 loggedIn = FALSE while (attempts < maxAttempts) AND (loggedIn == FALSE) username = input("Enter username") password = input("Enter password") if (username == correctUser) AND (password == correctPass) then print("Access granted") loggedIn = TRUE else attempts = attempts + 1 endif endwhile if loggedIn == FALSE then print("Account locked") endif
Input validation (no blank entries)	username = input("Enter username") password = input("Enter password") if (username.length == 0) OR (password.length == 0) then print("Username and password cannot be blank") else if (username == "admin") AND (password == "letmein") then print("Access granted") else print("Access denied") endif endif
Multiple users (simple “database” using arrays)	array users[3] array passwords[3] users[0] = "admin" passwords[0] = "letmein" users[1] = "teacher" passwords[1] = "classroom" username = input("Enter username") password = input("Enter password") found = FALSE for i = 0 to users.length - 1 if (username == users[i]) AND (password == passwords[i]) then found = TRUE endif

	next i if found == TRUE then print("Access granted") else print("Access denied") endif



Feature	What it means	Why it is useful	Example
Subprograms	Breaking a program into smaller parts (procedures/functions)	Makes code easier to read, test, and reuse	`procedure calculateTotal(price, quantity)`
Naming conventions	Using clear, meaningful names for variables and subprograms	Helps others understand what the code does	`studentScore` instead of `x`
Indentation	Lining up code to show structure	Makes selection and loops easier to follow	Code inside `if/while/for` is indented
Commenting	Adding notes to explain code	Helps explain complex	`// Check if the student has passed`

		logic and remind the programmer later	

2.3.1 Exam Questions on Defensive Design

Question	Tip to answer
1(a) What is input validation? **(1)** Input validation is checking data to make sure it meets the rules before a program accepts it.	"Check input meets rules."
1(b) Name one type of validation check. **(1)** Range check / Length check / Presence check.	Just name one.
1(c) What is authentication? **(1)** Authentication is checking a user's identity (e.g., username and password).	"Prove who you are."
1(d) Tick **one**: Which is usually more secure? **(1)** **A** Blacklist **B** Whitelist **B**	Whitelist = only allowed inputs.
2(a) State two ways a user could misuse an input box. **(2)** Enter letters when a number is needed.[1] Enter a very long string / symbols to try to break the program.[2]	Two short, different examples.
2(b) State two benefits of commenting code. **(2)** Makes code easier to understand for you/others.[1] Makes maintenance/debugging easier later.[2]	Think "understand" + "edit later".
2(c) A system accepts ages **0–120**. Give one **boundary** test value and one **invalid** test value. **(2)** Boundary: 0 (or 120).[1] Invalid: -1 (or 121).[2]	Boundary = edge. Invalid = outside.
3(a) Explain the difference between input sanitisation and input validation. **(3)** Sanitisation removes unwanted characters (cleans the input).[1] Validation checks the input matches rules (format/range/length).[2] Both help stop errors and reduce security risks.[3]	Sanitise = clean. Validate = check rules.
3(b) Explain why limiting login attempts improves security. **(3)** It reduces brute-force guessing of passwords.[1] After a few fails, the account can lock or delay attempts.[2] This makes it much harder for attackers to gain access.[3]	Mention brute force.
3(c) Explain how indentation and naming conventions improve maintainability. **(3)** Indentation shows which code is inside each IF/loop, so logic is clearer.[1] Good names (e.g., `totalTickets`) make the purpose obvious.[2] This helps others edit code safely and reduces mistakes.[3]	Readability + fewer bugs.
4(a) A program accepts whole numbers **1–100**. Write OCR Exam Reference Language to input `num`, and output **"accepted"** if valid, otherwise **"not accepted"**. **(4)** `num = int(input("Enter a number between 1 and 100"))`[1] `if num >= 1 AND num <= 100 then`[2] `print("accepted")`[3] `else`[4] `print("not accepted")`[3] `endif`	Two comparisons joined by AND. Include ELSE.
4(b) Write a **procedure** `showError()` that outputs **"INVALID INPUT"**, then call it. **(4)** `procedure showError()`[1] `print("INVALID INPUT")`[2] `endprocedure`[3] `showError()`[4]	Procedure = no return. Define then call.
4(c) Write a **function** `addVAT(price)` that returns `price * 1.2`. Then call it for `price = 50` and store the result in `total`. **(4)**	Function must RETURN. Use parameter.

Question	Tip to answer
`function addVAT(price)[1] withVAT = price * 1.2[2] return withVAT[3]endfunctiontotal = addVAT(50)[4]`	
6(a) A login system stores the correct password in `storedPass = "Synergy1"`. The user must enter a password. The program must: 1) remove spaces from the input, 2) allow **3 attempts**, 3) output **"Access granted"** if correct, otherwise **"Access denied"**, 4) after 3 failed attempts output **"Account locked"**. Write an algorithm (OCR Exam Reference Language). **(6)** `storedPass = "Synergy1"[1]attempts = 0[2]access = False[3]while attempts < 3 AND access == False[4] userPass = input("Enter password")[5] userPass = userPass.replace(" ","")[6] if userPass == storedPass then[1] print("Access granted")[2] access = True[3] else[4] print("Access denied")[5] attempts = attempts + 1[6] endifendwhileif access == False then print("Account locked")endif`	Count attempts. Loop. Only lock after 3 fails.
6(b) A ticket system starts with `tickets = 500`. A customer can book **1–10** tickets at a time. The program must: 1) ask for `num`, 2) validate `num` is 1–10, 3) if enough tickets remain, book them and subtract from `tickets`, otherwise output **"Not enough tickets"**, 4) repeat until `tickets` is 0. Write an algorithm (OCR Exam Reference Language). **(6)** `tickets = 500[1]while tickets > 0[2] num = int(input("How many tickets (1 to 10)?"))[3] if num < 1 OR num > 10 then[4] print("Invalid amount")[5] else if tickets >= num then[6] print("Tickets booked")[1] tickets = tickets - num[2] else[3] print("Not enough tickets")[4] endif endifendwhile`	Validate 1–10 first, then check tickets >= num, loop until 0.

2.3.2 Testing

2.3.2 Testing	
☐ The purpose of testing ☐ Types of testing: ○ Iterative ○ Final/terminal ☐ Identify syntax and logic errors ☐ Selecting and using suitable test data: ○ Normal ○ Boundary ○ Invalid/Erroneous ☐ Refining algorithms	**Required** ✓ The difference between testing modules of a program during development and testing the program at the end of production ✓ Syntax errors as errors which break the grammatical rules of the programming language and stop it from being run/translated ✓ Logic errors as errors which produce unexpected output ✓ Normal test data as data which should be accepted by a program without causing errors ✓ Boundary test data as data of the correct type which is on the very edge of being valid ✓ Invalid test data as data of the correct data type which should be rejected by a computer system ✓ Erroneous test data as data of the incorrect data type which should be rejected by a computer system ✓ Ability to identify suitable test data for a given scenario ✓ Ability to create/complete a test plan

TESTING (OCR J277)

What is testing?

Testing is checking that a program **works correctly, gives the right results,** and **does not crash.**

Purpose of testing

- Testing is used to:
  - Find errors and bugs
  - Check the program meets the requirements
  - Make sure the **program** works with **different inputs**
  - Improve **reliability** and **quality**

Testing helps prevent problems before users see them.

Testing during development vs final testing

Testing modules during development

This is testing **small parts** of the program as they are built.

- Done **while coding**
- Tests individual modules / subprograms
- Easier to **find where** errors occur
- Used to **fix problems early**

Example: Testing a login check before adding menus.

Testing at the end of production

This is testing **the whole program** together.

- Done after the program is finished
- Checks the program works as a complete system
- Uses realistic data

Example: Testing a full student system from login to saving results.

Types of testing

1. **Iterative testing** means testing little and **often** during **development.**
   - **Test** after **each change**
   - Errors are caught **early**

Most common type during coding.

Final (terminal) testing

2. **Tests the whole system**
   - Uses **normal, extreme,** and **invalid data**
   - Confirms the program is ready for use

Happens at the ***end*** of development.

Programs can contain **errors** that stop them working properly.

The two main types you need to know are **syntax errors** and **logic errors**. A **syntax error** is a mistake that **breaks the rules of the programming language** A **logic error** is when the program **runs**, but gives the **wrong or unexpected output**.

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes

OCR-style example	`if score >= 50`
	`  print("Pass")`
	`endif`
		`if score >= 50 then`
		`  print("Pass")`
		`else`
		`  print("Pass")`
		`endif`

What happens	Program will not run because `then` is missing	Program runs but always prints `"Pass"`
Common mistakes	Missing `then`, `endif`, brackets, or quotation marks	Using the wrong condition or same output in both choices

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes
OCR-style example (FOR loop)	for i = 1 to 5 print(i)	for i = 1 to 5 print(i)next i
What happens	Program will not run because `next i` is missing	Program runs but produces the wrong result
Common mistakes	Missing `next i`; incorrect FOR syntax	Using wrong start/end values; incorrect step value

Feature	Syntax error	Logic error
What it means	Breaks the grammatical rules of the programming language	The program follows the rules but the logic is wrong
Does the program run?	❌ No	✔️ Yes
OCR-style example (WHILE loop)	while count < 5 print(count) endwhile	count = 1 while count <= 5 print(count) endwhile
What happens	Program will not run because `count` was never defined	Program runs forever because `count` never changes
Common mistakes	Missing `count = 1` before the loop; missing `endwhile`	Forgetting `count = count + 1`; using a condition that never becomes FALSE

Testing – practise different boundaries: An integer input accepts values from 12 to 60 inclusive. Classify 35, 12, 60, 61 and "twelve" as normal, boundary or invalid/erroneous where appropriate. Explain why the exact minimum and maximum are valid boundary tests, but values just outside the range must be rejected.

SELECTING AND USING TEST DATA (OCR J277)

What is test data?

Test data is input used to check that a **program works correctly** and safely.

Good testing uses **different types** of data to check all situations.

1 Types of test data

Normal test data:

- is **valid**
- is **expected**
- Should be **accepted** by the program
- Should not cause errors



Used to check the program works in normal conditions.

1 Normal test data

- Breaks (f **valid**
- The smallest or largest **valid value**
- Right on the **edge** of what is **allowed**
- Still **valid** and should be accepted

Used to check the program works in normal conditions.

2 Boundary test data

- **The smallest or largest valid value**
- Right on the **edge** of what is allowed
- Still **valid** and **should be accepted**

Used to check the program handles limits correctly.

3 Invalid test data

- The **correct data type.**
- But outside the allowed range
- Should be **rejected** by the program

Used to check **validation** rules.

Choosing suitable test data

1. **Identify the valid range.**
2. Pick at least one **normal value.**
3. **Pick both boundary values.**
4. Pick **one invalid** and one **erroneous** value.

Test data is input used to check that a program works correctly.

Remember BIN...

- **B** **Boundary** – edges
- **1** **Invalid** – wrong value
- **E** **Erroneous** – wrong type

One-sentence exam question

Test data is input used to check that a program works correctly and **handies** errors safely.

2.3.2 Exam Questions on Testing

Question	Tip to answer
1(a) What is testing? **(1)**	"Check it works."
Testing is checking a program works correctly and finds errors before it is used.	
1(b) Name one type of testing. **(1)**	Just name one.
Iterative / Final (alpha) / Final (beta).	
1(c) What is a syntax error? **(1)**	"Breaks the rules of the language."
A syntax error is when code breaks the rules of the programming language (e.g. missing a bracket).	
2(a) State two reasons why programs are tested. **(2)**	Two clear reasons.
To find and fix errors/bugs.[1]To make sure the program meets the requirements / works as expected.[2]	
2(b) A program accepts whole numbers **1–100**. Give one **normal**, one **boundary**, and one **invalid/erroneous** test value. **(3)**	Normal = typical. Boundary = edge. Invalid = outside.
Normal: 27.[1]Boundary: 1 (or 100).[2]Invalid/erroneous: 0 (or 101 / "Hello").[3]	
2(c) State two differences between alpha testing and beta testing. **(2)**	Alpha = in-house. Beta = real users.
Alpha is done by developers/testers before release.[1]Beta is done by real users in real situations after alpha.[2]	
3(a) Explain why boundary test data is useful. **(3)**	Edges often break code.
Boundary data checks the extreme valid values.[1]It finds off-by-one mistakes (e.g. using < instead of <=).[2]This helps ensure validation rules are correct.[3]	
3(b) Explain the difference between a logic error and a runtime error. **(3)**	Logic = wrong result. Runtime = crashes while running.
A logic error means the program runs but gives the wrong output.[1]A runtime error happens while the program is running and may crash (e.g. divide by zero).[2]Logic errors are found by testing outputs; runtime errors are triggered by certain inputs/conditions.[3]	
4(a) A program asks for an age (0–120). Write OCR Exam Reference Language to: input `age`, validate the range, and output **"accepted"** or **"not accepted"**. **(4)**	Two comparisons joined by AND.
`age = int(input("Enter age (0 to 120)"))`[1] `if age >= 0 AND age <= 120 then`[2] `  print("accepted")`[3] `else`[4] `print("not accepted")`[3] `endif`	
4(b) A programmer fixes a bug, then tests again straight away. Name the testing approach and explain why it is used. **(4)**	Name + why (repeat improve).
Iterative testing.[1]After each change, you re-test to check the fix works.[2]It also checks you haven't created new bugs elsewhere.[3]This improves reliability before final release.[4]	
6(a) A program should accept a username that is **6–12 characters** long and not blank. Write an algorithm (OCR Exam Reference Language) that: inputs `username`, uses a loop to keep asking until it is valid, then outputs **"Username accepted"**. **(6)**	Use WHILE with OR conditions.

Question	Tip to answer

```
username = input("Enter username")[1]
while username.length < 6 OR username.length > 12[2]
  print("Invalid username")[3]
  username = input("Enter username")[4]
endwhile[5]
print("Username accepted")[6]
```

6(b) A program asks for a number **1–100**. Create a test plan table (in words) that includes **at least 2 normal**, **2 boundary**, and **2 invalid/erroneous** test values, and what the expected outcome is for each. **(6)**

Include expected result every time.

Normal: 27 → accepted.[1]
Normal: 85 → accepted.[2]
Boundary: 1 → accepted.[3]
Boundary: 100 → accepted.[4]Invalid/erroneous: 0 → not accepted.[5]
Invalid/erroneous: 101 (or “Hello”) → not accepted.[6]

2.4 Boolean Logic

Keyword	Definition
AND	Has two inputs and outputs 1 only if both are 1.
OR	Has two inputs, if both are 1 then outputs 1, otherwise outputs 0.
NOT	Has a single input and outputs the opposite of that input.
Truth Tables	A table which shows the possible input and output values for a logic gate.
Logic Gate	An operator such as AND, OR or NOT which performs a logical function on the input.
Logic Diagram	A diagram showing one or more logic gates together.

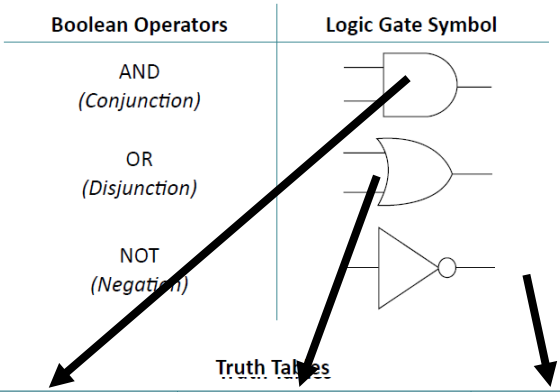
2.4.1 Boolean Logic

2.4.1 Boolean logic

- ☐ Simple logic diagrams using the operators AND, OR and NOT
- ☐ Truth tables
- ☐ Combining Boolean operators using AND, OR and NOT
- ☐ Applying logical operators in truth tables to solve problems

Required

- ✓ Knowledge of the truth tables for each logic gate
- ✓ Recognition of each gate symbol
- ✓ Understanding of how to create, complete or edit logic diagrams and truth tables for given scenarios
- ✓ Ability to work with more than one gate in a logic diagram

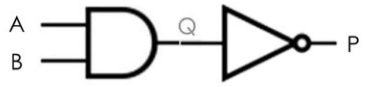


Truth Tables

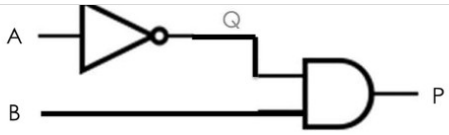
AND			OR			NOT	
A	B	A AND B	A	B	A OR B	A	NOT A
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Alternatives

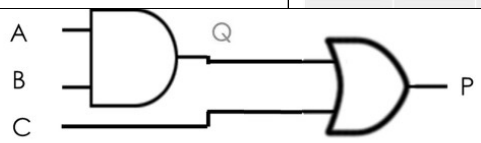
- Use of other valid notation will be accepted within the examination, e.g. Using T/F for 1/0, or V for OR, etc.



A	B	Q = A AND B	P = NOT (A AND B)
1	1	1	0
1	0	0	1
0	1	0	1
0	0	0	1

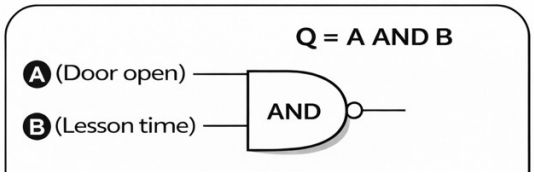


A	B	Q = Not A	P = (NOT A) AND B
1	1	0	0
1	0	0	0
0	1	1	1
0	0	1	0

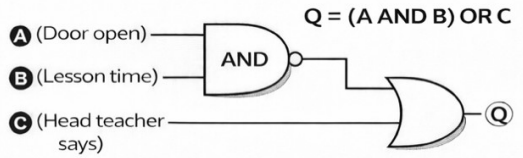


A	B	C	Q = A AND B	P = (A AND B) OR C
1	1	1	1	1
1	1	0	1	1
1	0	1	0	1
1	0	0	0	0
0	1	1	0	1
0	1	0	0	0
0	0	1	0	1
0	0	0	0	0

The alarm sounds only if the door is opened AND it is during lesson time



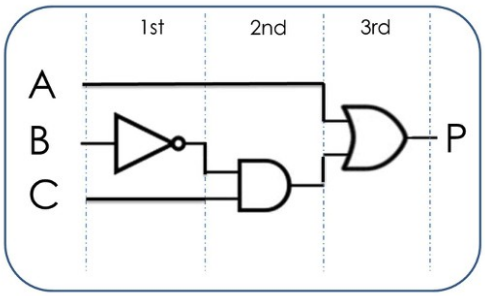
The alarm sounds if the door is opened AND it is during lesson time OR the head teacher says



- **A** = Door is open
- **B** = Break time
- **C** = Headteacher override switch is pressed

The alarm (P) should sound **if the door is open AND it is NOT break time**, **OR** if the headteacher override switch is pressed

So P = ((NOT B) AND C) OR A



2.4.1 Exam Questions on Boolean Logic

Question	Tip to answer
1(a) Name one Boolean operator. **(1)** AND / OR / NOT.	Just name one.
1(b) What output does a NOT gate produce? **(1) 0 -> 1 1->0**	"Flips it."
1(c) How many rows are in a truth table with 3 inputs? **(1)** 8	Use (2^n).
2(a) Complete the truth table for **P = A AND B**. **(2)**	Only 1 when both 1.

A B → P:
0 0 → 0.[1]
0 1 → 0.[2]
1 0 → 0.[1]
1 1 → 1.[2]

2(b) Complete the truth table for **Q = A OR B**. **(2)**	1 if either is 1.

0 0 → 0.[1]
0 1 → 1.[2]
1 0 → 1.[1]
1 1 → 1.[2]

2(c) Complete the truth table for **R = NOT A**. **(2)**

A=0 → R=1.[1]

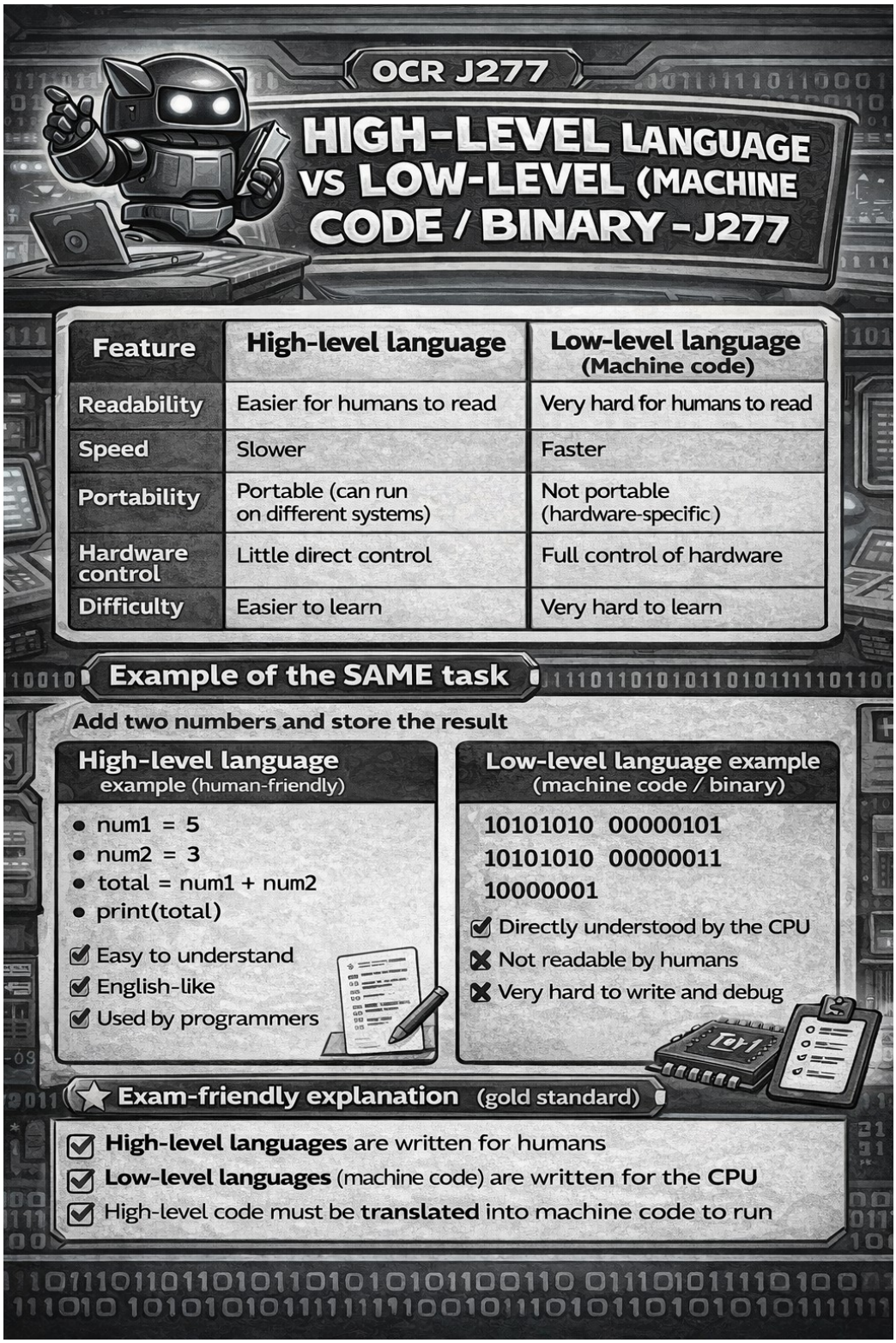
Question	Tip to answer
A=1 → R=0.[2]	
3(a) A security system has inputs: **A** = correct password, **B** = correct PIN, **C** = card blocked. Output **P** is True when the user enters the correct password **OR** correct PIN, **AND** the card is **NOT** blocked. Write the Boolean expression for P. **(3)**	Brackets + NOT C.
P = (A OR B) AND NOT C.[123]	
3(b) Explain why the brackets are needed in **P = (A OR B) AND NOT C**. **(3)**	Order matters.
Brackets force A OR B to be worked out first.[1]Without brackets, the system might combine the wrong parts first and change the meaning.[2]This could allow/deny access incorrectly.[3]	
4(a) A logic circuit has inputs A and B going into an AND gate. The AND output then goes into a NOT gate to produce output P. Write the expression for P. **(4)**	AND then NOT.
4(b) Complete the truth table for **P = NOT(A AND B)**. **(4)**	AND first, then flip.
00: A AND B =0 → P=1.[1] 01: 0 → 1.[2] 10: 0 → 1.[3] 11: 1 → 0.[4]	
6(a) A cinema app lets a user in if: they are **12 or older** AND they have **paid**. Variables: `age` (integer), `paid`(Boolean). Write an algorithm (OCR Exam Reference Language) that outputs **"ALLOW"** or **"DENY"**. **(6)**	Two checks joined by AND.
age = int(input("Enter age")) [1]paid = input("Paid? Y/N")[2] if paid == "Y" AND age >= 12 then[3] print("ALLOW")[4] else[5] print("DENY")[6] endif	
6(b) A game unlocks a reward if the player has **either** collected a key (**K**) **or** completed the quest (**Q**), and they are **not banned** (**B**). Write the Boolean expression, then complete the truth table for all 3 inputs. **(6)**	Use (K OR Q) AND NOT B.
Expression: P = (K OR Q) AND NOT B.[1] Truth table rows (KQB → P): 000 → 0.[2] 001 → 0.[3] 010 → 1.[4] 011 → 0.[5] 100 → 1.[6] 101 → 0.[6] 110 → 1.[6] 111 → 0.[6]	

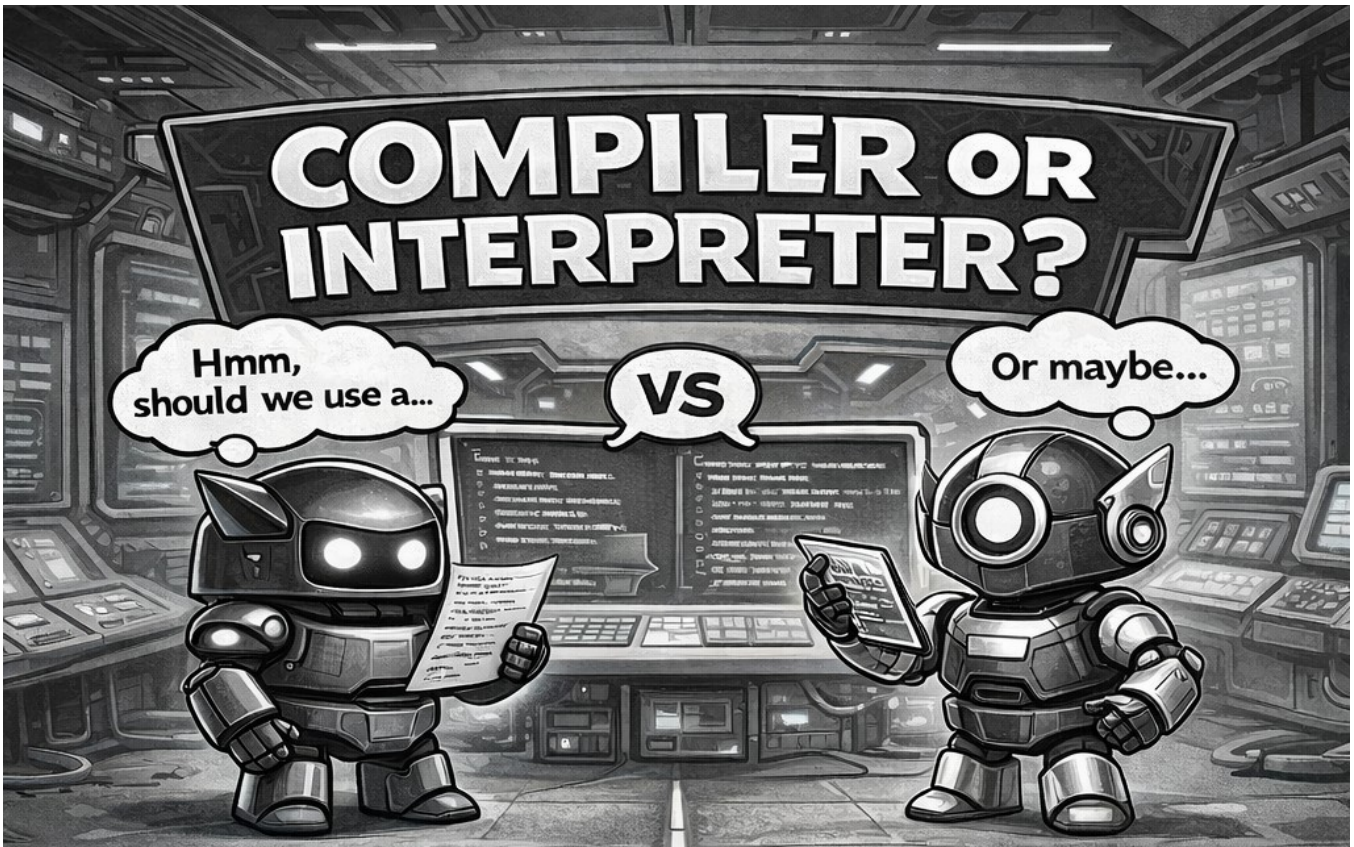
2.5 Programming Languages and Integrated Development Environments

Keyword	Definition
Programming Language	A language which can be understood by computers to produce computer software.
High Level Language	A programming language which is close to English in the commands used making it easier to learn and understand.
Low Level Language	A programming language closer to machine code, using less English like commands making it harder to understand.
Translator	A piece of software which converts programming language into machine code which the computer can understand and execute.
Compiler	A translator which converts code in one go.
Interpreter	A translator which converts code line by line.
Integrated Development Environment	A piece of software used by developers to write computer code. They often contain features such as translators and syntax highlighters to make it easier to write code.
Editor	A piece of software used to write computer code.
Error Diagnostics	Tools to help locate and resolve errors in code.
Runtime Environment	A tool which allows code to run on a system for which it was not designed. For example, running a mobile phone app on a desktop computer.

2.5.1 Languages

2.5.1 Languages	
☐ Characteristics and purpose of different levels of programming language: ○ High-level languages ○ Low-level languages ☐ The purpose of translators ☐ The characteristics of a compiler and an interpreter	**Required** ✓ The differences between high-level and low-level programming languages ✓ The need for translators ✓ The differences, benefits and drawbacks of using a compiler or an interpreter **Not required** × Understanding of assemblers





COMPILER VS INTERPRETER

COMPILER

- **What it does**
- Translates the **whole program** in one go
- Produces **an executable file**
- Reports **all errors** at once
- Program runs fast after compilation

Characteristics

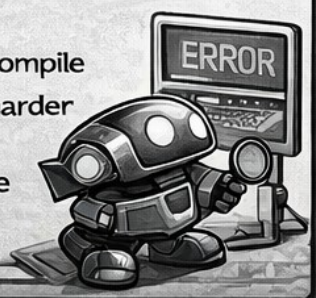
- Translates the whole program in onego
- Produces an executable file
- Reports all errors at once
- Program runs fast after compilation

Benefits

- Faster execution
- Errors shown together
- No translator needed once compiled

Drawbacks

- Takes time to compile
- Errors may be harder to locate
- Must recompile after changes



INTERPRETER

- **What it does**
- An interpreter **translates** and runs the program **line by line**
- Translates one line at a time.
- No executable file created.

Benefits

- Easier to debug
- Errors shown one at a time
- Good for learning and testing

Drawbacks

- Slower execution
- Translator needed every time
- Errors only shown as they occur

- **Drawbacks**
- Slower execution
- Translator needed every time



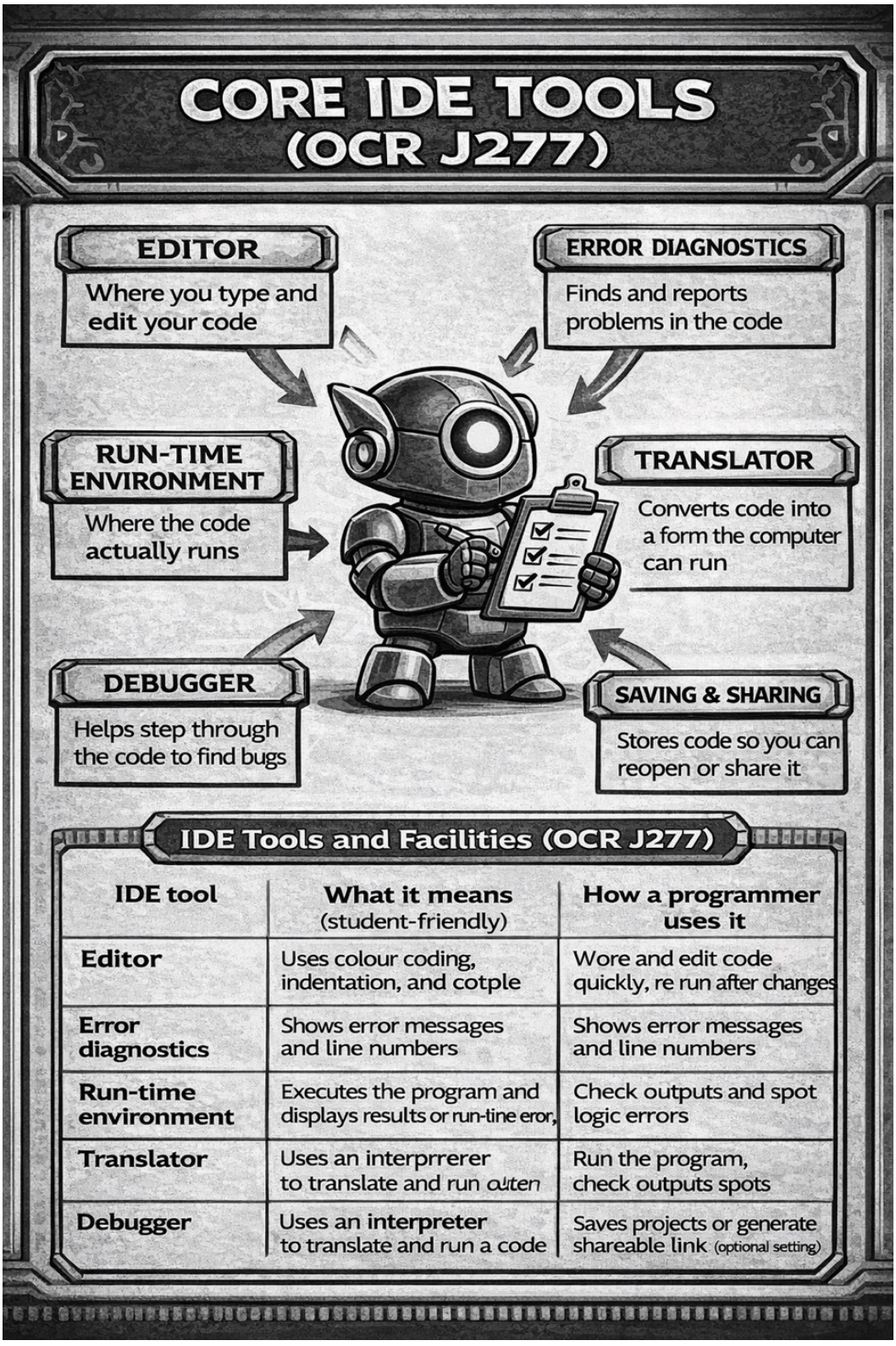
2.5.1 Exam Questions on Languages

Question	Tip to answer
Which type of language is easiest for humans to read and write? (1) ☐ Machine code ☐ Assembly ☐ High-level language ☐ Hexadecimal	Humans
High-level language.[1]	
Which translator converts high-level code into machine code before running the program? (1) ☐ Interpreter ☐ Compiler ☐ Assembler ☐ Debugger	"Before running
Compiler.[1]	
State one reason why machine code is difficult for humans to use. (1)	It's 1s and 0s?
It is written in binary (1s and 0s), so it is hard to read and write.[1]	
Explain the difference between a high-level language and a low-level language. (3)	3 points: readability, hardware dependence, translation.
High-level languages use English-like keywords so they are easier for humans to read and write.[1] Low-level languages (like assembly/machine code) are closer to the CPU instructions and are harder for humans.[2] High-level code must be translated into machine code before the CPU can run it.[3]	
Explain why translators are needed. (3)	CPU only understands machine code.
The CPU can only understand machine code (binary).[1] Translators convert high-level code into machine code so the program can run.[2] Without a translator, the CPU would not know what the high-level instructions mean.[3]	
Compare a compiler and an interpreter. (4)	2 points each: how they translate + pros/cons.
A compiler translates the whole program into machine code before it runs.[1] This usually makes the program run faster once compiled.[2] An interpreter translates and runs the code line by line.[3] This makes it easier to spot errors quickly because it stops where the error happens.[4]	
A company is creating software for a portable games console. Recommend whether they should use a compiler or an interpreter, and justify your choice. (6)	Use TIER style: pick one, give benefits + drawback.
A compiler is best for a portable games console because the program can be translated into machine code before it is shipped.[1] This means the game will run faster and more smoothly on the device.[2] It also means the console does not need to translate code while playing, saving processing power and battery.[3] A compiled program can be easier to distribute because users just run the executable.[4] However, compiling takes longer when developing because you must recompile after changes.[5] Overall, the faster performance is the key reason for using a compiler on a console.[6]	

2.5.2 The Integrated Development Environment (IDE)

2.5.2 The Integrated Development Environment (IDE)	
☐ Common tools and facilities available in an Integrated Development Environment (IDE): ○ Editors ○ Error diagnostics ○ Run-time environment ○ Translators	**Required** ✓ Knowledge of the tools that an IDE provides ✓ How each of the tools and facilities listed can be used to help a programmer develop a program ✓ Practical experience of using a range of these tools within at least one IDE

IDE tools help programmers **write code faster, find errors more easily,** and **improve program quality.**



IDE tool	What it means	How the IDE provides it	How a programmer uses it	Common exam mistakes
Editor	Where you type and edit your code	Uses colour coding, indentation, and autocomplete	Write and edit code quickly, re-run after changes	Saying the editor ***runs*** code (it doesn't — it only edits)
Error diagnostics	Messages that explain what went wrong and where	Shows error messages and line numbers	Find the error, fix it, run again	Confusing error diagnostics with the debugger

Run-time environment	Where the program runs and shows output	Executes the program and displays results or run-time errors	Check outputs and spot logic errors	Forgetting that logic errors appear here, not as syntax errors
Translator	Converts code into a form the computer can execute	Uses an **interpreter** to translate and run code	Test code behaviour exactly as it would execute	Saying the IDE ***is*** the translator rather than ***contains*** one
Debugger	Helps find errors by stepping through code	Runs code line by line and shows variable values	Track how values change to find bugs	Saying it only finds syntax errors (it's mainly for logic errors)
Sharing / saving work	Allows programs to be saved or shared	Saves projects or generates shareable links	Reopen work later or share with others	Forgetting this is a ***useful feature***, not a core programming tool

2.5.2 Questions on the Integrated Development Environment (IDE)

Question	Tip to answer (student)
State one type of error that can happen in a program. (1)	Any one: syntax / logic / runtime.
Syntax error.[1]	
Which error happens when the program breaks while it is running (e.g. divide by zero)? (1) ☐ Syntax error ☐ Runtime error ☐ Logic error ☐ Semantic error	"While running" = runtime.
Runtime error.[1]	
Explain what a syntax error is. (2)	2 points: breaks language rules + stops running/compiling.
A syntax error is when code breaks the rules of the language (wrong keyword/symbol/indentation).[1] The program will not compile or run until it is fixed.[2]	
Explain what a logic error is. (2)	2 points: runs but wrong output.
A logic error is when the program runs but the algorithm is wrong.[1] This means it produces the wrong output/results.[2]	
A student writes: `if score >= 5 AND <= 10 then` Explain why this causes an error and correct it. (3)	Must compare the variable twice.
It is wrong because <= 10 is not a full comparison (it is missing the variable).[1] The computer cannot tell what should be compared to 10.[2] Correct example: if score >= 5 AND score <= 10 then[3]	
Describe two IDE tools that help find and fix errors. (4)	2 tools + what each does.
Debugger (breakpoints/stepping) lets you run code line-by-line to see where it goes wrong.[1] Variable watch shows the values of variables while the program runs so you can spot incorrect values.[2] Syntax highlighting / error messages point out where syntax mistakes are in the code.[3] Autocomplete suggests correct keywords and reduces typing mistakes.[4]	
A program is meant to add up 1 to 10, but it outputs 45. Identify the likely type of error and explain how you would locate it using an IDE. (6)	Name error type + stepping + watch variables + check loop bounds.
This is most likely a **logic error** because the program runs but gives the wrong total.[1] I would use the debugger to step through the loop one line at a time.[2] I would add a breakpoint at the start of the loop so it stops each iteration.[3] I would use a variable watch to check the running total and the loop counter as they change.[4] I would check the loop bounds (e.g. starting at 0 or stopping too early) and the update line (e.g. adding the wrong value).[5] Once I find the line where the total becomes wrong, I would correct the calculation or the loop range and run again to test.[6]	

Resources

Common Exam Questions – worked through

Rewrite Flowchart (4 mark question)

(iii) The following flowchart outputs a message depending on how long each person has spent playing computer games.



Rewrite the flowchart as a program.
You must use **either**:

- OCR Exam Reference Language, **or**
- a high-level programming language that you have studied.

1 mark Use IF statement

1 mark Use Input

```
1 minutes = int(input("Enter minutes played"))
2 if minutes > 120 then
3     print ("You played games fortoo long!")
4 else
5     print ("You are under your time limit!")
6 endif
```

1 mark Print the correct value.

1 mark Print the correct value.

Draw Flowchart (4 mark question)

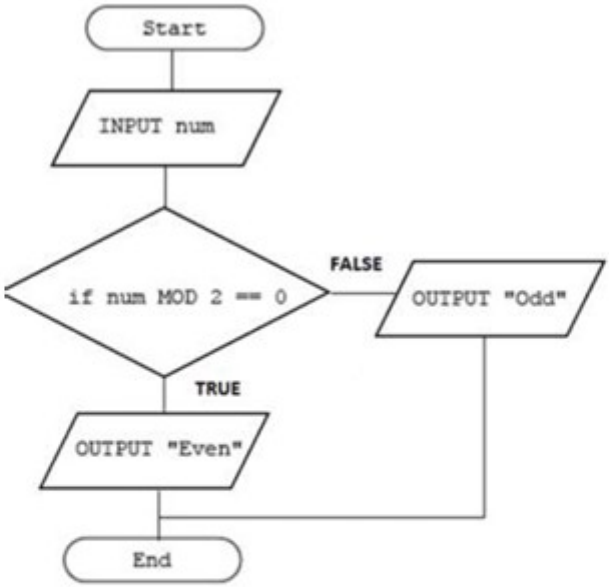
An algorithm decides if a number is odd or even. An odd number divided by 2 will give the remainder 1. The flowchart statements have been written for the algorithm, but the flowchart is incomplete. **Complete** the flowchart.



[4]

1 mark Correct shape for all three inputs AND outputs (Parallelogram)

1 mark Correct shape for decision(diamond)



1 mark True and False labelled correctly

1 mark All lines joined up AND correctly link to End

Write Function (8 mark question)

An algorithm stores the position of a character on a straight line as an integer. A user can move the character left or right. The following algorithm:

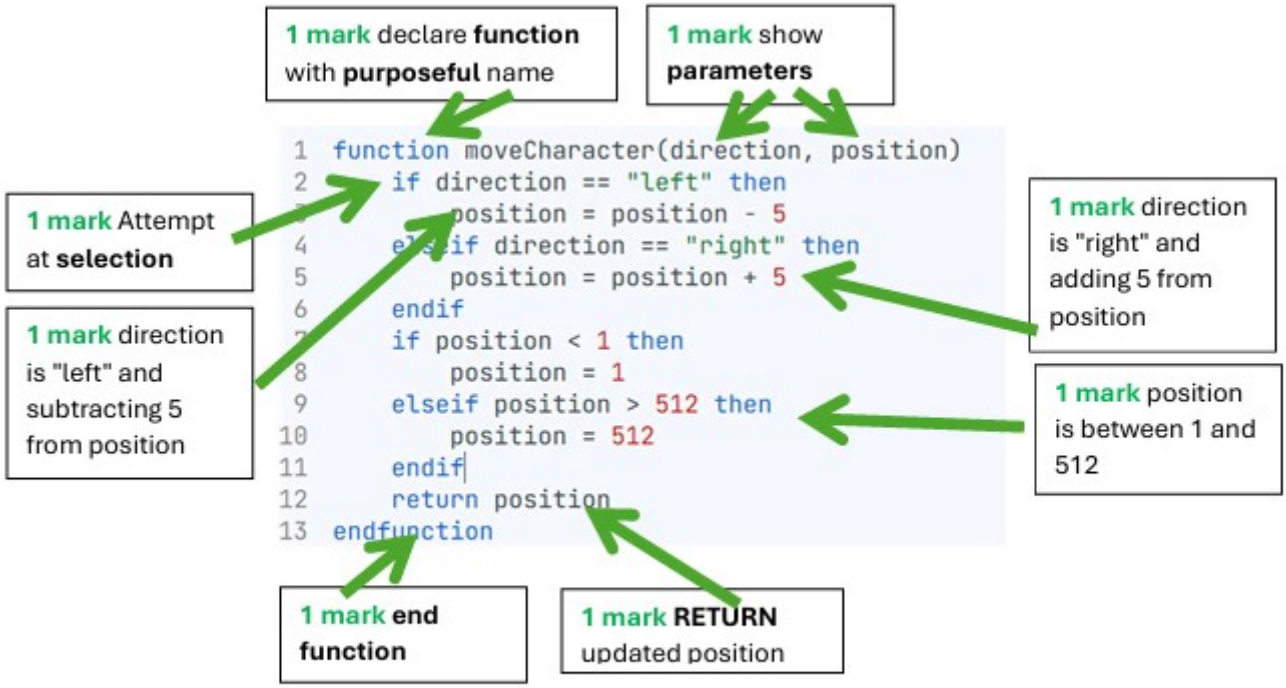
generates one random number between 1 and 512 (inclusive) to store as the position
prompts the user to input a direction to move (left or right)
prompts the user to input a direction to move (left or right)
takes a direction as input **until** a valid direction is input.

```
p = random(1, 512)
print("The position is ", p)
a = ""
while a != "left" AND a != "right"
    a = input("Enter direction, left or right")
endwhile
```

If the character moves left, 5 is subtracted from the position.
If the character moves right, 5 is added to the position.
The position of the character can only be between 1 and 512 inclusive.
The function moveCharacter():

- takes the direction (left or right) and current position as parameters
- changes position based on direction
- sets position to 1 if the new position is less than 1
- sets position to 512 if the new position is greater than 512
- returns the new position.

Write the **function** moveCharacter()



Understanding 2D Arrays (10 mark question)

SQL is all about getting information from tables (2D Arrays), however, SQL is **not** available for this GCSE challenge.

Ash is researching how long, to the nearest minute, each student in the class spends playing computer games in one week (Monday to Friday). Ash is storing the data in a 2D array called Gaming.

Days of the week	Students 0	1	2	3
0	60	30	45	0
1	180	60	0	60
2	200	30	0	20
3	60	10	15	15
4	100	35	30	45

i) Ash wants to output how much time is lost on **Friday** (Days of the Week **index** 4).

Create a program to calculate this total. **(4)**

ii) Ash wants to output how much time is lost all week in the class of 4!

Create **both** a visual flowchart and pseudocode algorithm to solve this problem **(6)**

Email answers to computingdept@reephamhigh.org.uk

The best answer receives a mystery prize!

Exam Paper 2 2026

OCR GCSE Computer Science (J277)
Paper 2: Computational Thinking, Algorithms and Programming
Time: 1 hour 30 minutes
Total Marks: 80

INSTRUCTIONS

- Use black ink.
- Answer **all questions**.
- **Section B** includes questions that must be answered using **OCR Exam Reference Language** or a high-level language you have studied.
- Calculators are not allowed.

Section A (40 marks)
Question 1 — Validating input (9 marks)

1(a) A program asks a user to enter a number. Only **integer values between 5 and 40 (inclusive)** are accepted.

Tick (✓) one or more boxes in each row to identify the type(s) of test data.

Example test data	Normal test data	Boundary test data	Invalid / erroneous test data
21	☐	☐	☐
"blue"	☐	☐	☐
46	☐	☐	☐
40	☐	☐	☐

[4]

1(b)(i) Complete the alternative algorithm.

```
fee = input("Enter a whole number between 5 and 40")

if fee _____________________ AND _____________________ then
    print("within range")
_____________________
    print("outside range")
endif
```

[3]

1(b)(ii) Give the identifier of **one variable** used in the new algorithm in part (i). **[1]**

1(b)(iii) Give **one Boolean operator** used in the algorithm in part (i). **[1]**

Question 2 — Constructs and Iteration (8 marks)

The following algorithm repeats while x is greater than zero: x = input("Enter a number"); while x > 0: print("GO"); x = x - 3; endwhile.

2(a) For each input, state how many times "GO" is output.

Input value	**Number of times "GO" is output**
4	_____
11	_____

[2]

2(b)(i) Identify and describe **two different examples of iteration** used in a high-level language. **[4]**

1

2

2(b)(ii) Give the names of **two other programming constructs** used in a high-level language. **[2]**

1 _____

2 _____

Question 3 — Searching and Sorting (9 marks)

3(a) Show the steps that a **merge sort** would take to put the data set into **ascending** order. **[4]**

Data set: `8 3 6 1 7 2 5 4`

3(b) Tick (✓) one box in each table. **[3]**

An **insertion sort**...

- ☐ uses an array with a sorted part and an unsorted part
- ☐ picks out the middle value to begin with
- ☐ merges arrays of numbers together

A **bubble sort** compares pairs of values and then:

- ☐ always swaps them
- ☐ swaps them if they are in the correct order
- ☐ swaps them if they are in the incorrect order

An **array**:

- ☐ is declared with a fixed length
- ☐ stores multiple values that cannot be changed while the program is running
- ☐ can only be declared with a maximum of one dimension (1D)

3(c) A searching algorithm:

- compares the first value in an array to the value being searched for
- stops if the comparison is true
- moves to the next value if the comparison is false

3(c)(i) Give the name of the searching algorithm described. **[1]**

3(c)(ii) Identify one other condition that will cause the algorithm to stop. **[1]**

Question 4 — File Handling and Casting (6 marks)

An algorithm reads scores from the external text file scores.txt.
Each score is output only if it is greater than 50.

Complete the algorithm using statements from the box.
Not all statements are used.

Choose the correct statements:
`open` `myFile.readLine()` `myFile.close()` `print` `input()` `temp` `txt` `x` `for` `if` `myFile.writeLine()`

4(a) Complete the algorithm. **[5]**

```
myFile = _____________________________("scores.txt")

while NOT myFile.endOfFile()
    temp = _____________________________
    if int( _____________________________ ) > 50
        _____________________________(temp)
    endif
endwhile
```

_____

4(b) Identify how **casting** is used in this algorithm. **[1]**

Question 5 — Boolean Logic and Computational Thinking (8 marks)

A security system has three inputs:

- **A** = True when the user enters the correct password
- **B** = True when the user enters the correct PIN
- **C** = True when the card is blocked

The output **P** is True if:

- the user enters **either** the correct password **or** the correct PIN
- and the card is **not** blocked

5(a) Draw the logic circuit for the system. **[3]**

(Blank space for circuit)

5(b) Complete the truth table for **P = A AND B**. **[2]**

A	B	P

5(c) Match each computational thinking technique to the correct example. **[3]**

Abstraction → _____

Decomposition → _____

Algorithmic thinking → _____

Section B (40 marks)

In this section, questions that assess writing or refining algorithms must be answered using either:

- **OCR Exam Reference Language**, or
- a **high-level programming language** you have studied.

Question 6 — Festival Ticket System Scenario (30 marks)

A festival system stores information about acts and ticket sales.

6(a) Password checking (4 marks)

A visitor enters a password twice. The passwords are stored in variables **passA** and **passB**.

6(a)(i) Tick (✓) one box showing the correct statements to store both passwords. **[1]**

- ☐ passA and passB = input("enter password twice")
- ☐ input("enter your password") = passA; input("enter again") = passB
- ☐ passA = input("enter your password"); passB = input("enter your password again")
- ☐ "passA" = input("enter your password"); "passB" = input("enter again")

6(a)(ii) Write an algorithm that:

- checks if **passA** and **passB** are the same
- outputs **"passwords match"** if they are the same
- outputs **"passwords do not match"** if they are not the same

[3]

6(b) SQL and data types (7 marks)

A database table `TblMusic` stores data about music acts:

ActID	Stage	Day	SetLength
1	Platinum	Saturday	0.5
2	Hills	Saturday	1.5
3	Triangle	Sunday	1.0
4	Platinum	Sunday	0.5

An SQL statement is intended to show **ActID and Stage** for all acts playing on **Saturday**, but it is incorrect:

```
SELECT ActID AND Stage
FROM TblActs
IF Day = Saturday
```

6(b)(i) Refine the SQL statement to correct the errors. **[4]**

6(b)(ii) Identify one example of a **record** from `TblMusic`. **[1].**

6(b)(iii) Give the most appropriate data type for each field. **[2]**

Stage: _____

SetLength: _____

6(c) Functions for festival merchandise (6 marks)

Festival programmes cost £18 each.

6(c)(i) Write a function **programmeCost()** that:

- takes the number of programmes as a parameter
- calculates and **returns** the total price

[4]

6(c)(ii) Write an algorithm that:

- uses **programmeCost()** to find the price for **5 programmes**
- stores the result in the variable **totalCost**

[2]

6(d) Trace tables and refinement (7 marks)

The number of tickets sold per day is stored in an array:

```
dailySold = [34, 28, 41]
```

Algorithm:

```
01 total = 0
02 for x = 0 to 2
03    total = total + dailySold[x]
04 next x
05 print(total)
```

6(d)(i) Complete the trace table. You may not need to use all rows. **[4]**

Line number	total	x	Output

The array changes to include **two more days**:

```
dailySold = [34, 28, 41, 30, 22]
```

6(d)(ii) Identify the line number to change and write the updated line. **[2]**

Line number: _____

Updated line: _____

6(d)(iii) Refine line 05 to output:
Total sold = followed by the total. **[1]**

New line 05: _____

6(e) Defensive design: festival merchandise loop (6 marks)

There are **125 wristbands available**. The number available is stored in **wristbands.**

Complete an algorithm that:

- prompts the user to enter how many wristbands they want
- outputs **"Wristbands sold"** and reduces wristbands if enough remain
- outputs **"Not enough wristbands"** if not enough remain
- repeats until there are **no wristbands left**

You must use OCR Exam Reference Language or a studied high-level language.

[6]

wristbands = 125

Question 7 — Languages and Translators (10 marks)

7(a) Describe the difference between a **compiler** and an **interpreter**. **[4]**

7(b) Explain why **translators are needed** for high-level languages. **[3]**

7© A student is practising programming in an online coding environment and wants **instant feedback** while they type and run code.

Recommend whether a **compiler** or an **interpreter** is more suitable, and give one benefit and one drawback. **[3]**

END OF QUESTION PAPER

Answers and Common misconceptions

Question	Right answer (student-friendly)	Common misconceptions
1(a)	**21 = Normal ✓ • "blue" = Invalid/erroneous ✓ • 46 = Invalid/erroneous ✓ • 40 = Boundary ✓** ***(also acceptable as normal + boundary because it's valid and at the edge)***	Saying **40 is only normal** (OCR often credits boundary here). Mixing up **invalid**with **boundary**.
1(b)(i)	`if fee >= 5 AND fee <= 40 then … else …`	Writing `fee >= 5 AND <= 40` (second comparison missing fee). Using `fee > 5 AND fee < 40` (wrong because it rejects 5 and 40).
1(b)(ii)	`fee`	Writing a ***value*** like `40` instead of a variable identifier.
1(b)(iii)	`AND`	Writing = instead of a Boolean operator. Writing && without understanding it means AND (depends on language).
2(a)	Input 4 → 2 times "GO" • Input 11 → 4 times "GO" ***(the body executes before subtracting 3 each time)***	Forgetting the initial value of the counter (often starts at 0). Confusing < with <= so you output one extra time.
2(b)(i)	Any two valid iterations + descriptions, e.g. **FOR** (count-controlled, runs a fixed number of times) and **WHILE** (condition-controlled, repeats while condition is true) OR **REPEAT UNTIL**(condition checked at end; runs at least once).	Saying "WHILE repeats until condition is true" (it repeats ***while*** true). Saying "loops repeat infinitely" (only if condition never changes).
2(b)(ii)	**Sequence** and **Selection** ***(or "sequencing" / "branching")***	Naming examples like "IF" instead of the construct name **selection**.
3(a)	Merge sort: split to singles → merge into sorted pairs → merge into sorted 4s → final sorted 8. Final order: 1 2 3 4 5 6 7 8	"Merge then sort" (merge sort does the sorting ***during*** merging). Sorting into descending when asked ascending.
3(b)	Insertion sort: ✓ **uses a sorted + unsorted part**. Bubble sort: ✓ **swaps if in the incorrect order**. Array: ✓ **declared with a fixed length**.	Thinking arrays can always grow (confusing with Python lists). Thinking bubble sort always swaps.
3©(i)	**Linear search**	Writing "binary search" (binary search starts in the middle and needs ordered data).
3©(ii)	Stops when it reaches the **end of the list / last index checked** and it's still not found.	Writing only "when not found" (not a checkable stopping condition unless you define "end reached").
4(a)	`myFile = open("scores.txt") temp = myFile.readLine() if int(temp) > 50 print(temp)` after loop: `myFile.close()`	Putting `close()` inside the loop. Using `writeLine()` instead of `print()`. Forgetting to cast before comparing to 50.
4(b)	Casting is used to **convert `temp` to an integer** so it can be compared to 50 (using `int(temp)`).	Saying "casting changes the value" (it changes the **data type**, not the meaning). Too vague: "it changes data type" without context.
5(a)	Logic circuit for: **P = (A OR B) AND (NOT C)**	Missing the NOT on C (so blocked cards still pass). Using AND instead of OR between A and B. Drawing NAND/NOR by adding circles incorrectly.
5(b)	Truth table for **A AND B**: 00→0, 01→0, 10→0, 11→1	Missing a combination (common when not "counting in binary"). Putting 1s in rows where only one input is 1.
5©	**Abstraction** = remove unnecessary	Mixing up abstraction and

Question	Right answer (student-friendly)	Common misconceptions
	detail. **Decomposition** = split problem into smaller parts. **Algorithmic thinking** = step-by-step rules to solve it. ***(Exact matches depend on the examples shown in the question.)***	decomposition ("abstraction = splitting"). Writing examples instead of naming the technique.
6(a)(i)	✓ `passA = input("enter your password"); passB = input("enter your password again")`	`input(...) = passA` (wrong direction). Using "`passA`" (string) instead of variable `passA`.
6(a)(ii)	Example: `if passA == passB then print("passwords match") else print("passwords do not match") endif`	Checking only one branch (no else). Using = for comparison in languages that require ==.
6(b)(i)	Correct SQL: `SELECT ActID, Stage FROM TblMusic WHERE Day = "Saturday"`	Using `AND` between fields (should be a comma). Using `IF` instead of `WHERE`. Forgetting quotes around Saturday. Wrong table name.
6(b)(ii)	Any full row, e.g. **(1, Platinum, Saturday, 0.5)**	Defining "a record" in words instead of giving an example row. Giving only one field instead of the whole row.
6(b)(iii)	Stage = **String/Text** • SetLength = **Real/Float/Decimal**	Saying "integer" for SetLength (it has decimals). Writing "variable" / "input" (not data types).
6©(i)	Function with parameter + return, e.g. `function programmeCost(quantity) price = quantity * 18 return price endfunction`	Asking for input inside the function (overwriting the*

Exam Paper 2 – 2026 (Version B)

OCR GCSE Computer Science (J277)
Paper 2: Computational Thinking, Algorithms and Programming
Time: 1 hour 30 minutes
Total Marks: 80

Answer all questions.
Section A (40 marks): short/structured questions (you may use pseudocode / structured English where helpful).
Section B (40 marks): programming and algorithm questions. **You must answer using either the OCR Exam Reference Language or a high-level language you have studied.**

Section A (40 marks)
Question 1 – Computational Thinking (6 marks)

1(a) Define **decomposition**. **[1]**

1(b) Define **abstraction**. **[1]**

1© A teacher wants a program to manage a **school club sign-up** list. Students enter: name, year group, and whether they have permission (Y/N).

State **two examples of abstraction** the programmer could apply. **[2]**

1 _____

2 _____

1(d) Explain how **pattern recognition** could help when designing the club sign-up solution. **[2]**

Question 2 – Programming Constructs and Trace Tables (8 marks)

A program uses this algorithm:

```
01 total = 0
02 x = 1
03 while x <= 5
04     total = total + x
05     x = x + 2
06 endwhile
07 print(total)
```

2(a) Complete the trace table. You may not need to use all rows. **[4]**

Line number	total	x	Output

2(b) Identify the **type of iteration** used (count-controlled or condition-controlled). **[1]**

2© Identify **two other programming constructs** used in high-level languages. **[2]**

1 _____

2 _____

2(d) Give the **final output** produced by line 07. **[1]**

Question 3 – Searching and Sorting (Insertion sort focus) (8 marks)

3(a) A programmer chooses between **linear search** and **binary search**.
State **one requirement** for binary search to work. **[1]**

3(b) Explain why binary search is often faster than linear search on large datasets. **[2]**

3(c) The list below is sorted using **insertion sort** into ascending order.

Data: **[7, 3, 9, 4]**

Show the key steps of an insertion sort (the sorted and unsorted parts). **[4]**

Step 1: _____

Step 2: _____

Step 3: _____

Step 4: _____

3(d) One advantage of insertion sort compared to bubble sort on nearly-sorted data. **[1]**

Question 4 – Data Types and Casting (6 marks)

4(a) State the most appropriate data type for each item. **[3]**

Item	Data type
A student's surname	_____
A ticket price of 12.50	_____
A login success flag (True/False)	_____

4(b) A user enters their age using `input()`. Explain why **casting** may be needed before the age can be compared to a number (e.g. `>= 13`). **[2]**

4(c) Write one line of code/pseudocode that casts a variable called `ageText` into an integer called `age`. **[1]**

Question 5 – Strings, ASCII and Common Built-ins (6 marks)

Assume:

```
word = "ComputerScience"
```

5(a) Give the value of `word.length` (or equivalent). **[1]**

5(b) Give the output of `word.left(8)` (or equivalent). **[1]**

5(c) Give the output of `word.substring(3, 4)` where the first number is the start index and the second is the number of characters. **[1]**

5(d) Explain the difference between **CHR(66)** and **ASC('B')**. **[2]**

5(e) A common mistake is treating `"7"` as the same as `7`. Explain one problem this can cause in a program. **[1]**

Question 6 – Boolean Logic (6 marks)

6(a) Complete the truth table for: **P = (A OR B) AND (NOT A)** **[3]**

A	B	P
0	0	
0	1	
1	0	
1	1	

6(b) Draw a logic circuit for: **Q = (A AND B) OR (NOT C) [3]**
(Use correct AND/OR/NOT gate symbols.)

Section B (40 marks)

Answer using OCR Exam Reference Language or a high-level language you have studied.

Question 7 – Defensive Design: Validation + Authentication + Maintainability (14 marks)

A system allows users to log in with a username and password.

7(a) Write an algorithm that performs a **presence check** so the username cannot be blank. **[3]**

7(b) Write an algorithm that performs a **length check** so a password must be **8 to 20 characters** (inclusive). **[4]**

7(c) Write a simple authentication algorithm that:

- stores the correct password in `storedPass`
- asks the user for a password in `enteredPass`
- allows **3 attempts** then outputs `LOCKED OUT`
- outputs `ACCESS GRANTED` if correct **[5]**

7(d) State **two maintainability improvements** a programmer could apply to this codebase. **[2]**

1 _____

2 _____

Question 8 – Arrays / 2D Arrays + SQL-style thinking (14 marks)

A school stores behaviour points for students. A 2D array called `points` is used:

- Each **row** is a student
- Each **column** is a week (Week 1 to Week 4)

Example:

```
points = [ [2, 1, 0, 3], [0, 0, 1, 1], [5, 2, 2, 0] ]
```

8(a) Identify the value at `points[2][1]`. **[1]**

8(b) Write an algorithm to calculate and output the **total points for student 0** (the first row). **[4]**

8(c) Write an algorithm to find the **highest single weekly value** anywhere in the 2D array. **[5]**

8(d) A database table `TblPoints(StudentName, Week, Points)` could store the same data.
Write an SQL statement to show `StudentName` and `Points` for records where `Week = "Week 3"`. **[4]**

Question 9 – File Handling + Refining Code (12 marks)

A text file `scores.txt` stores one score per line. Example:

```
12
0
45
9
0
```

9(a) Complete an algorithm that reads each line, and outputs only scores that are not 0. **[6]**
(Use file handling + casting where needed.)

9(b) The program is changed so it must also count how many non-zero scores were printed, then output:
`Total non-zero scores = X`
Write the additional lines needed. **[4]**

9(c) Identify **one line** where casting is needed and explain why. **[2]**

Line: _____

Explanation: _____

END OF QUESTION PAPER

Answers and Common misconceptions

Question	Right answer (student-friendly)	Common misconceptions
1(a) Decomposition	Breaking a big problem into smaller, manageable parts.	Saying it means “removing details” (that’s abstraction).
1(b) Abstraction	Removing unnecessary details and keeping only what matters.	Thinking it means “splitting the problem up” (that’s decomposition).
1(c) Abstraction examples	Any 2: ignore exact phone model; ignore student’s full address; store year group as 7/8/9 etc not tutor group; store permission as Y/N not a full paragraph; don’t store how they travelled to school.	Listing extra features instead of removing detail; keeping everything “just in case”.
1(d) Pattern recognition	Spotting repeated steps/data (e.g., all sign-ups need name + year + permission) so you can reuse the same checks/inputs each time.	Confusing it with “guessing patterns” or “making a loop” only.
2(a) Trace table	Key states: start total=0, x=1 → after loop runs: total=1 x=3 → total=4 x=5 → total=9 x=7 → output 9.	Updating x before adding; stopping at x=5 without doing the final add; forgetting the loop repeats while condition is true.
2(b) Iteration type	**Condition-controlled** (WHILE).	Calling it count-controlled just because it repeats a few times.
2(c) Other constructs	**Sequence** and **Selection**.	Writing examples like “IF” or “WHILE” (these are statements/keywords, not the named constructs).
2(d) Final output	9	Thinking it outputs 1+3+5+7 (=16) because they forget it stops at x<=5.
3(a) Binary search requirement	The data must be **sorted/ordered**.	Saying “it needs an array” (linear search also uses arrays/lists).
3(b) Why binary search is faster	It halves the search space each step (divide and conquer), so fewer checks on big lists.	Saying “because it starts in the middle” without explaining the halving; claiming it works on unsorted data.
3(c) Insertion sort steps	Example steps: Sorted [7] + Unsorted [3,9,4] → insert 3: [3,7] → insert 9: [3,7,9] → insert 4: [3,4,7,9].	Doing swaps like bubble sort; not showing sorted/unsorted parts; sorting descending by accident.
3(d) Insertion vs bubble advantage	Insertion sort is usually quicker on nearly-sorted data (fewer moves).	Saying “always faster” (depends on the data).
4(a) Data types	Surname = **String**; 12.50 = **Real/Float**; True/False flag = **Boolean**.	Writing “text” as a type without matching exam wording; mixing up char vs string.
4(b) Why casting needed	`input()` usually gives text, so you must convert to an integer before numeric comparisons like `>= 13`.	Saying casting is “encrypting” or “formatting”; comparing strings to numbers directly.
4(c) Cast line	Example: `age = int(ageText)`	Using == instead of assignment; writing `int(age)` when `age` doesn’t exist yet.
5(a) Length	`"ComputerScience"` length = **15**.	Counting wrong (forgetting it’s 15); mixing up length with last index (14).
5(b) left(8)	**"Computer"**	Returning 7 letters; starting from index 1 instead of 0.
5(c) substring(3,4)	Starting at index 3 for 4 chars gives **"pute"**.	Treating the second number as an end index; off-by-one errors.
5(d) CHR vs ASC	`CHR(66)` turns number → character (**'B'**). `ASC('B')` turns character → number (**66**).	Swapping them around; thinking ASCII is “encryption”.
5(e) "7" vs 7 problem	`"7" + "1"` could become `"71"` not 8, or comparisons behave wrongly.	Assuming the computer “knows what you mean” and auto-converts safely every time.

Question	Right answer (student-friendly)	Common misconceptions
6(a) Truth table for P=(A OR B) AND (NOT A)	Rows (A,B→P): 00→0, 01→1, 10→0, 11→0.	Forgetting NOT A; doing OR last; writing 1 for 11 because "OR makes it true".
6(b) Circuit for Q=(A AND B) OR (NOT C)	AND gate on A,B; NOT gate on C; OR gate combines those two outputs to Q.	Drawing NAND/NOR by adding circles; putting NOT on the whole expression instead of just C.
7(a) Presence check username	Loop until username isn't blank (e.g., while length==0 ask again).	Checking only once (no loop); checking `username > 0` (wrong for strings).
7(b) Length check password 8–20	Accept if `len(pass) >= 8 AND len(pass) <= 20`, else reject/re-ask.	Writing `len(pass) >= 8 AND <= 20`(missing variable/len on second condition); using < and excluding 8 or 20.
7(c) Authentication 3 attempts	Use a counter attempts=0; while attempts<3 and not correct: ask; if match → ACCESS GRANTED; after 3 fails → LOCKED OUT.	Resetting attempts inside loop; printing ACCESS GRANTED but not stopping; allowing unlimited tries.
7(d) Maintainability improvements	Any 2: meaningful variable names, consistent indentation, comments, subroutines, constants, blank lines for blocks.	Saying "add more features"; confusing maintainability with security only.
8(a) points[2][1]	**2**	Mixing up row/column order; using 1-based indexing and answering 5.
8(b) Total for student 0	Sum row 0: 2+1+0+3 = **6** (loop through columns, add to total, output).	Summing a column instead of a row; starting total as 1; missing one value.
8(c) Highest value anywhere	Set `max` to first element, nested loops through all rows/cols; if current > max then max=current; output max (here **5**).	Setting max=0 and failing if negatives existed; only checking one row; forgetting nested loop.
8(d) SQL for Week 3	`SELECT StudentName, Points FROM TblPoints WHERE Week = "Week 3";`	Using AND between fields instead of comma; using IF instead of WHERE; missing quotes around Week 3.
9(a) Read file + print non-zero	Open file; while not EOF: read line; cast to int; if !=0 then print; end; close file.	Comparing `"0"` as a string incorrectly; forgetting to close the file; using writeLine instead of readLine.
9(b) Count printed non-zero + final message	Add `count=0`; when printing increment count; after loop print `Total non-zero scores = ` + count.	Incrementing count for zeros too; printing count inside loop only; forgetting to initialise count.
9(c) Where casting is needed	Where you compare to 0: e.g., `if int(line) != 0` because file lines are text.	Saying "casting is optional"; vague answer like "it changes type" with no context.

Useful QR Codes

Revision QR Poster – Scan to Revise

OCR Official Past Papers & Examiner Reports

Craig 'n' Dave – OCR J277 Videos

BBC Bitesize – OCR GCSE CS

Isaac Computer Science – OCR GCSE

Physics & Maths Tutor – OCR GCSE

Seneca Learning – GCSE CS